



OO Analysis & Design

Using

Unified Modeling Language



Chapter (1)

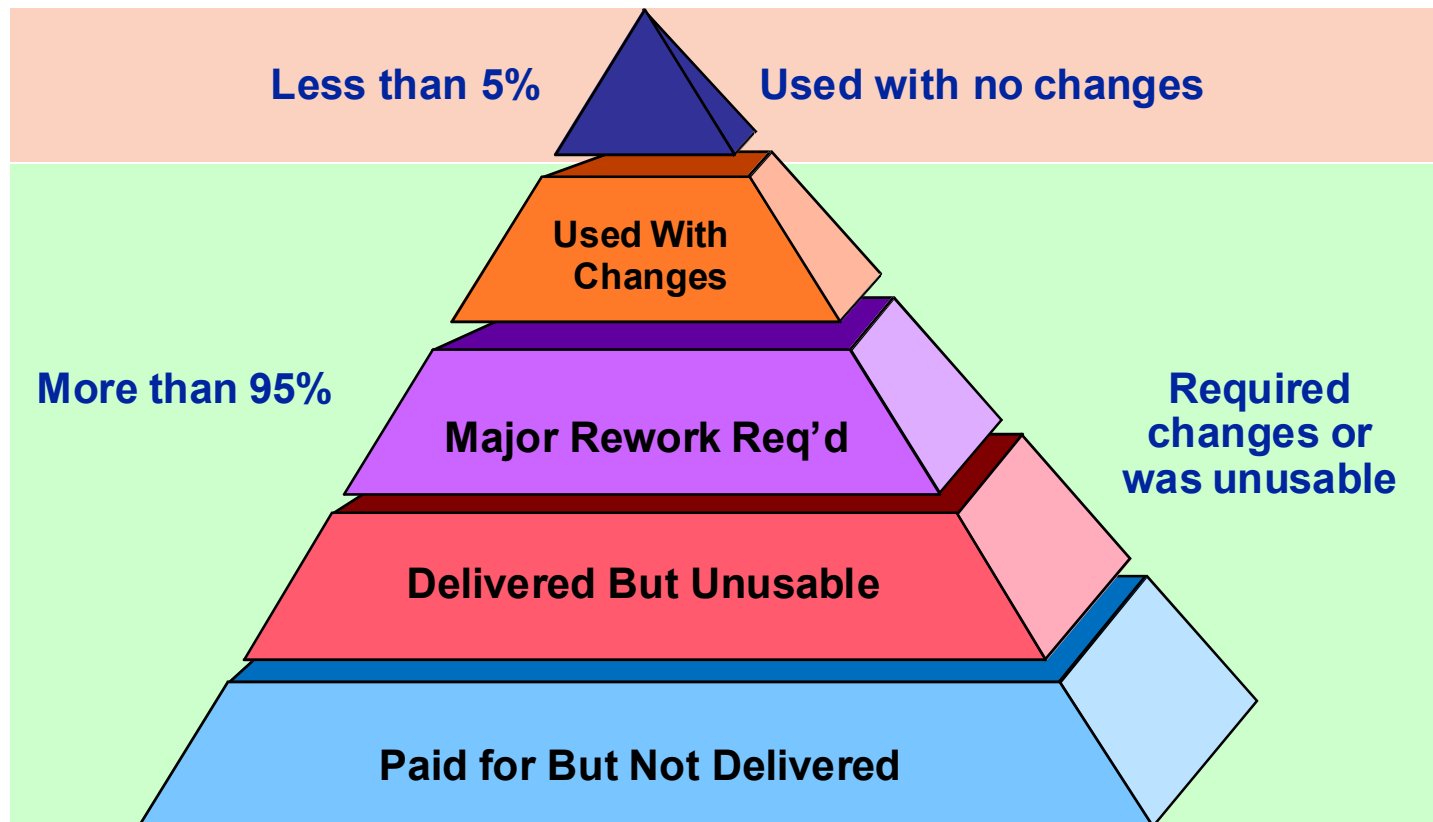
Introduction



The Software Crisis

**Software failures reported
by W. Wayt Gibbs
In the Sep 1994
issue of Scientific American**

The Software Crisis



Source: US. Government Data

The Software Crisis

- California department of motor vehicles
 - spent over **\$43 million** on a system
 - To merge the state's driver and vehicle registration systems
 - ✓ System was abandoned without ever being used



The Software Crisis

- American Airlines

- Lost a **\$165 million**



- To link its flight-reservation software with the reservation systems of Marriott, Hilton, and Budget

The Software Crisis

- The Denver Airport

- Lost **millions of dollars** due to the delay of the opening of the airport
 - The delay happened because of the computerized baggage system of the airport.



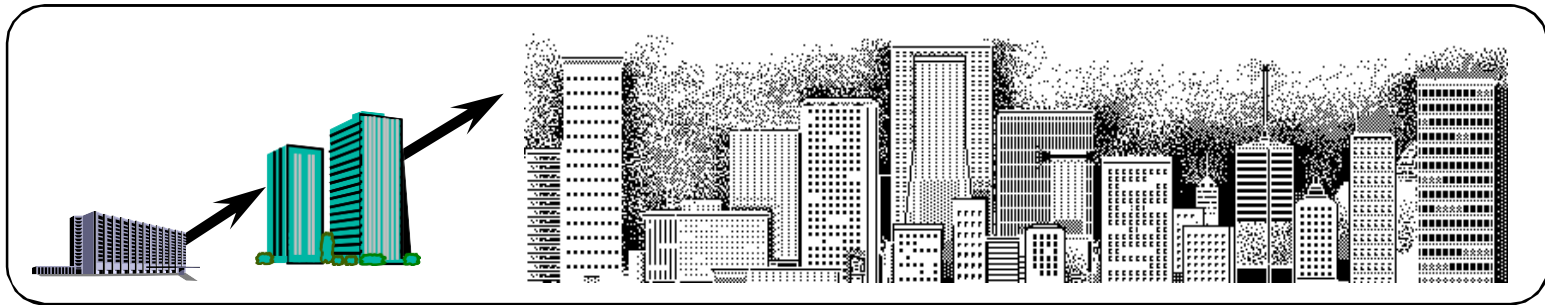
The Software Crisis

- What are the reasons for the software crisis?
 1. Rolling baseline
 2. Failure to manage risk
 3. Software complexity



1. Rolling Baseline

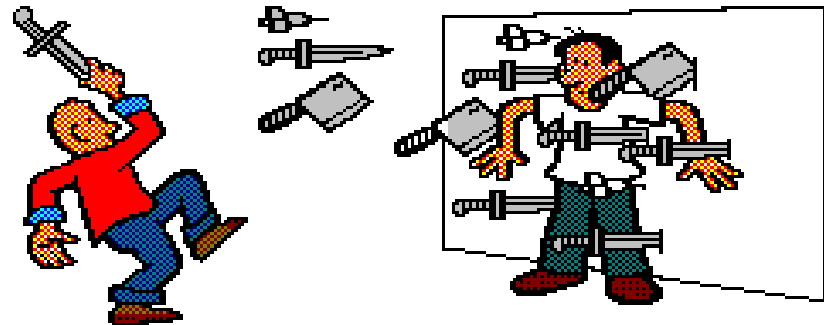
- Shorter development cycles are business requirements
 - Users expect more in terms of flexibility
- Initial requirements usually poorly defined





2. Failure to Manage Risk

- The waterfall lifecycle can delay problem identification
- There is no proof that the system will run until late in the lifecycle
- The result is maximum risk





3. Software Complexity

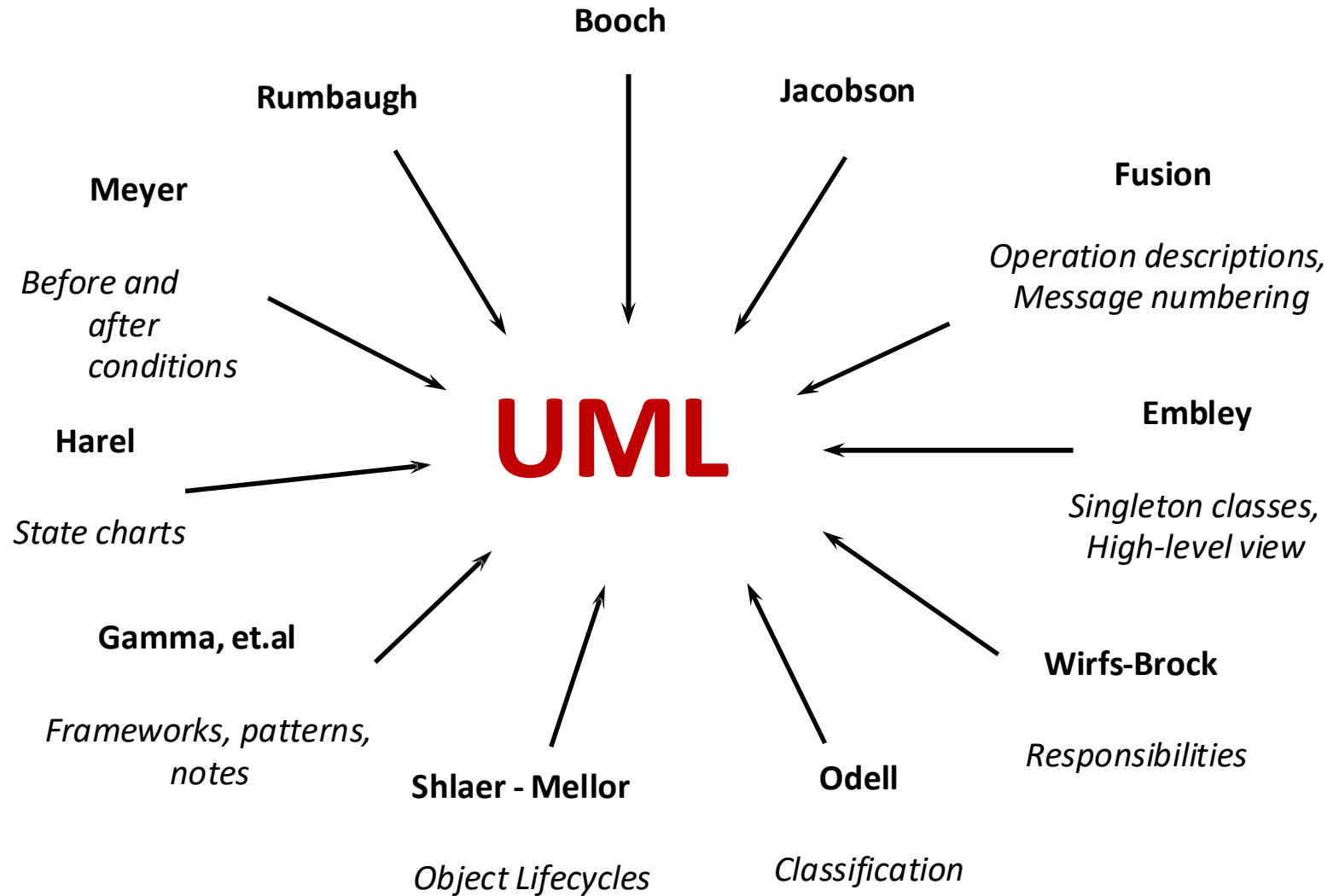
- Business software demands are increasing
- No one understands the entire system
- Legacy systems must be maintained, but the original developers are gone



What is the UML ?

- ***Unified Modeling Language*** (UML):
 - Is the standard visual modeling language
 - It provides us with a syntax for describing and designing software systems.
- Through to its core, UML leans towards object oriented software development.

Inputs to UML





Inputs to UML

In 1997,

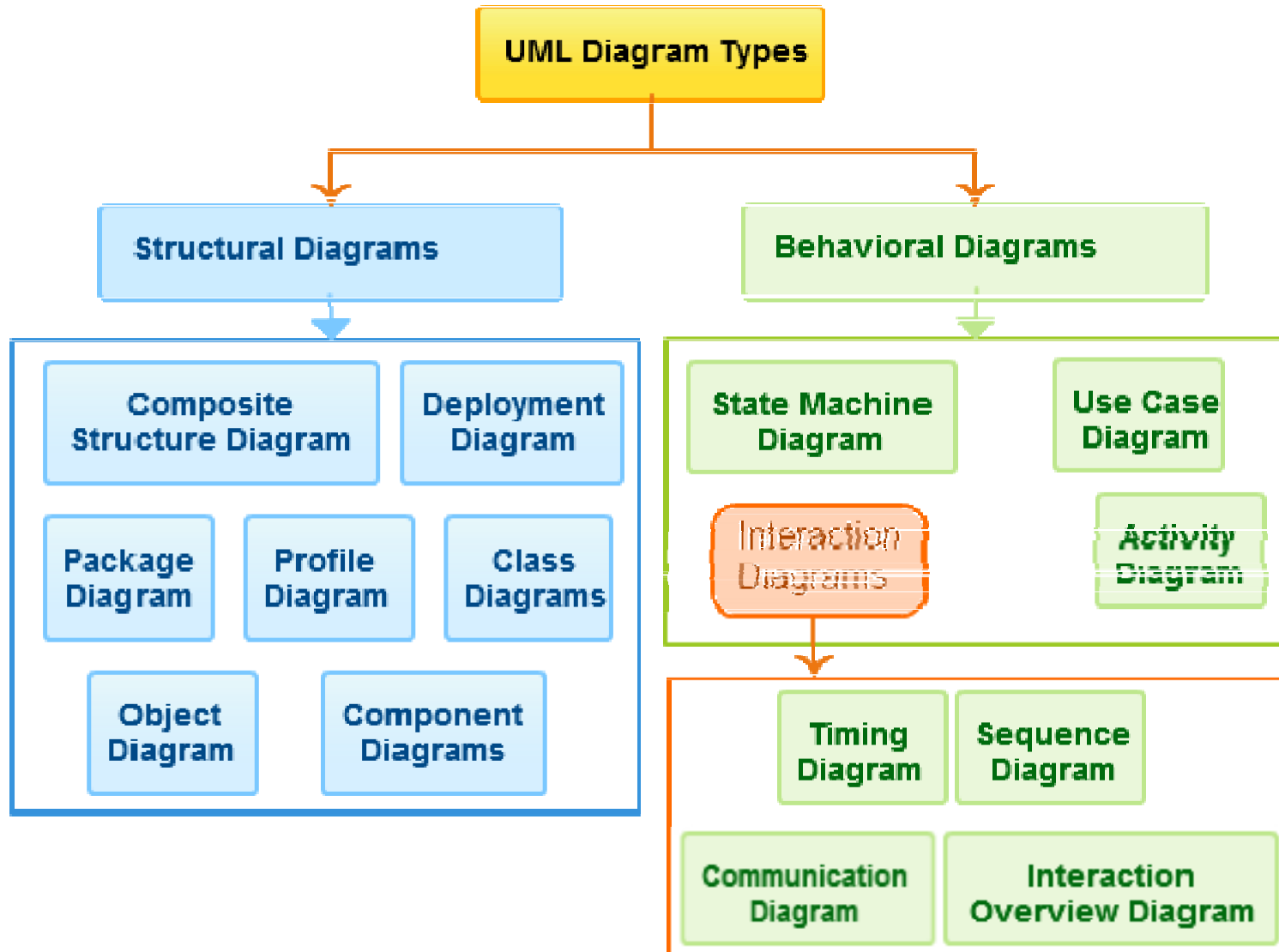
The UML was adopted by the OMG,

And since then

The OMG have owned and maintained the language.

OMG are the Object Management Group, an industry wide, non profit making standards body.

UML Diagram types





Benefits of the UML

- Defines a seamless mapping from analysis to design to implementation
- Defines an expressive and consistent notation
 - Makes it easier to communicate with others
 - Helps point out omissions and inconsistencies
 - Supports both small and large-scale analysis and design



Chapter (2)

A Development Process

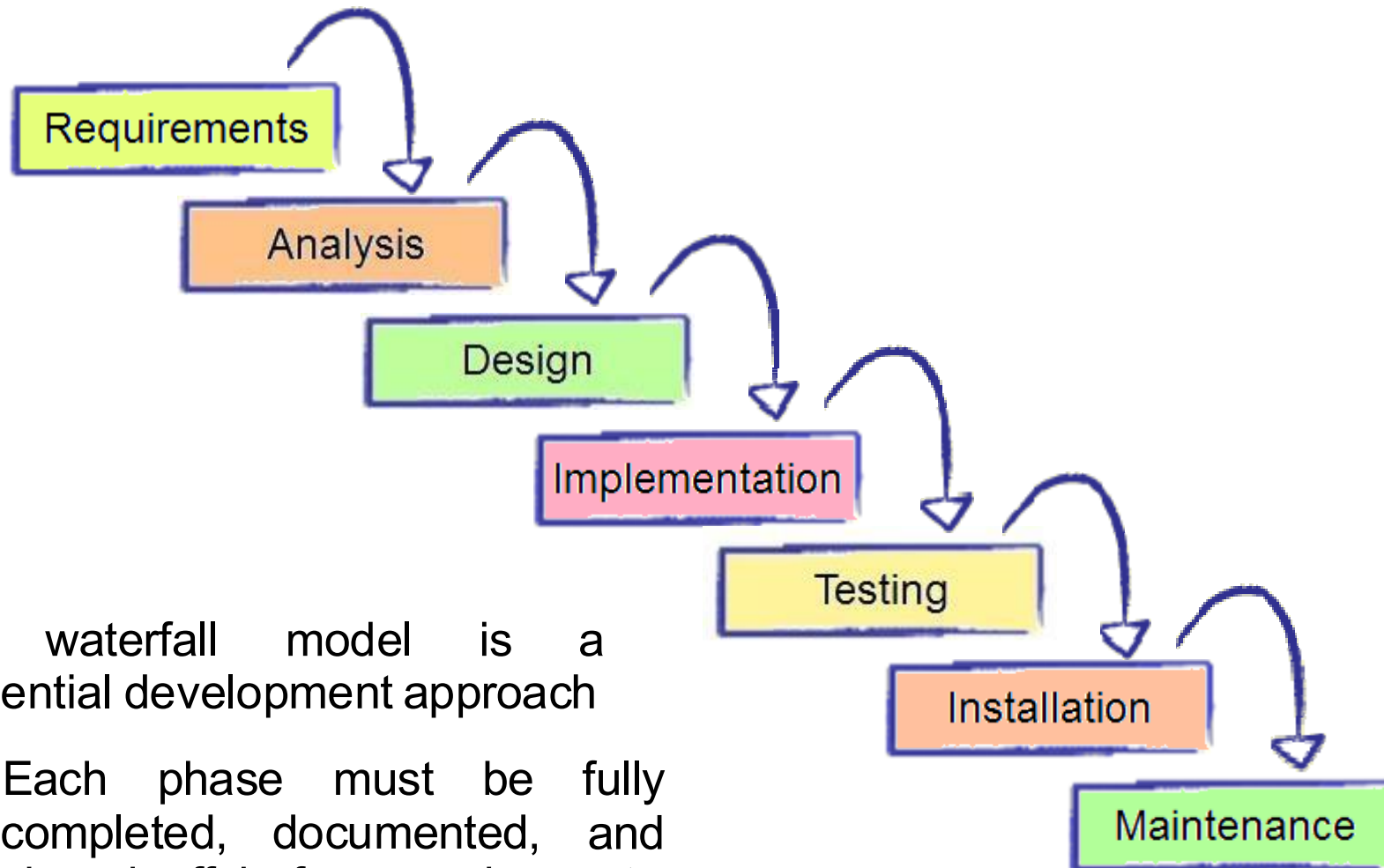


UML as a Notation

UML is simply a language, a notation, a syntax,
whatever you want to call it.

It does not tell you
How to develop software.

1. Waterfall model



The waterfall model is a sequential development approach

- Each phase must be fully completed, documented, and signed off before moving onto the next phase.



1. Waterfall model (**Advantages**)

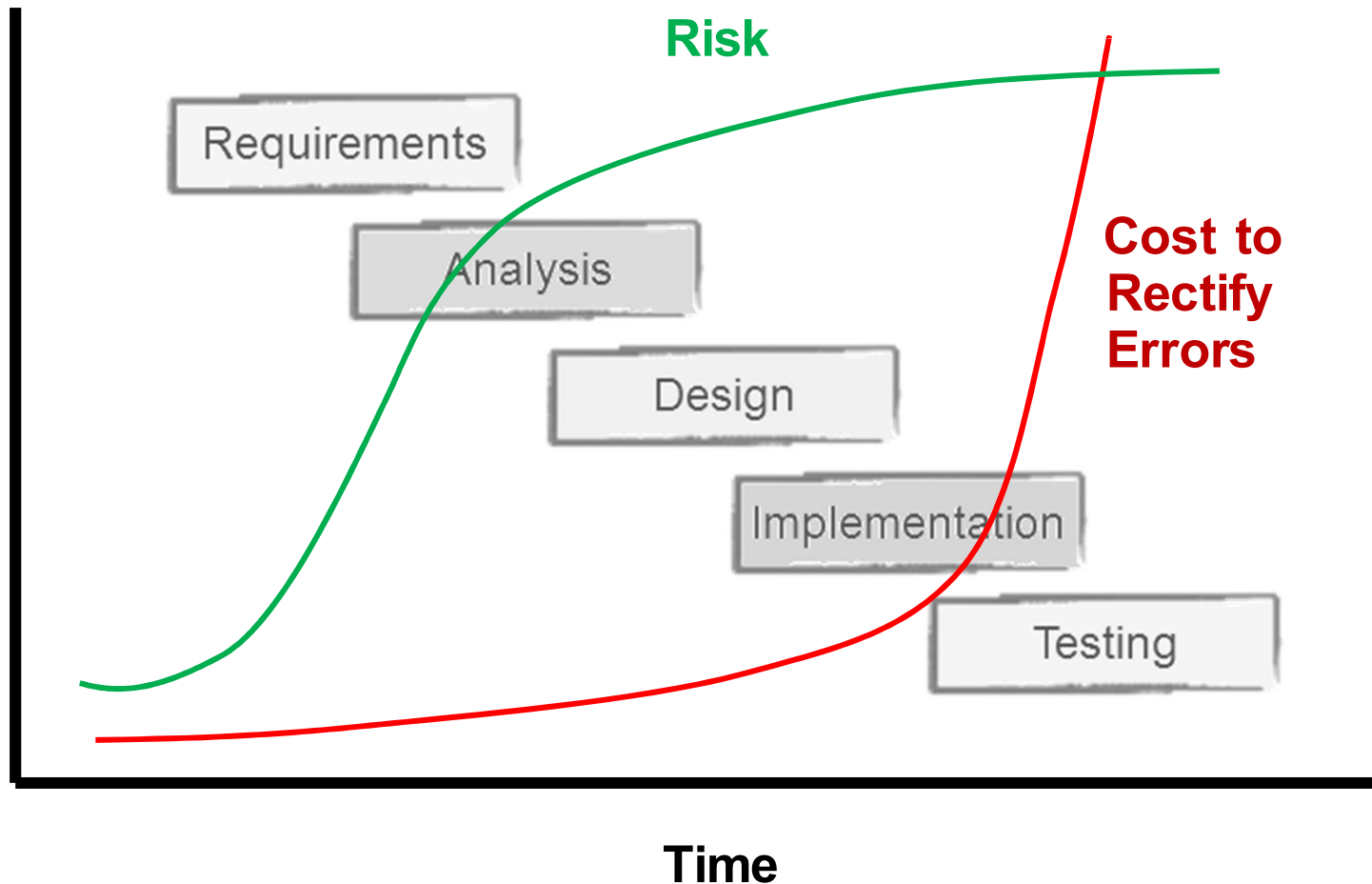
1. If in the beginning of the project failures are detected, it takes less effort (time and money) for this error.
2. In the waterfall model lay the emphasis on documentation.
3. Authorities and responsibilities can be clearly delineated.
4. The way of working ensures that there are specific phases.
 - Every project phase can be carefully planned
5. One can use this method of milestones.
 - The waterfall model is well known. So there might be easy to work.



1. Waterfall model (**Disadvantages**)

1. Everything is on the critical path.
2. It takes too long to see results.
3. It depends on stable, correct requirements.
 - Often the requirements over the project change.
4. The phases are very large, it is very difficult to estimate how much each step cost.
5. It is not efficient for smaller projects.
 - There's too much effort to the project itself around in terms of documentation.

1. Waterfall model





1. Waterfall model

In Summary,

The waterfall model is easy to understand and simple to manage.

But

the advantages of the model being to break down once the complexity of the project increases.



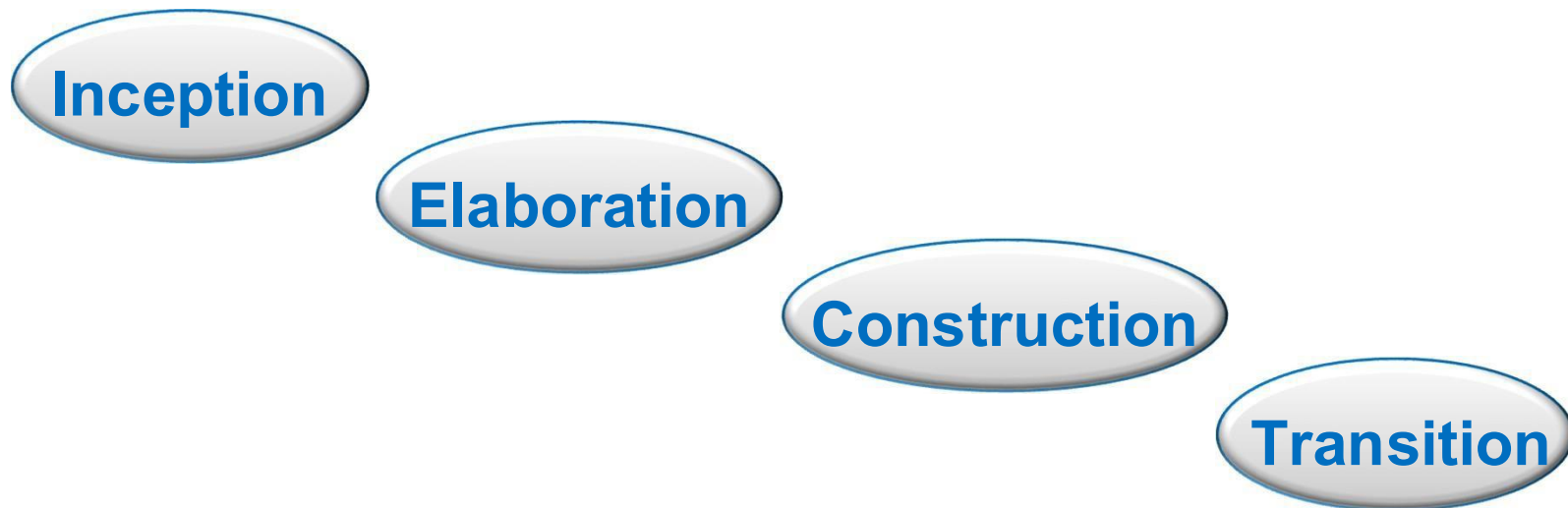
2. Iterative Model

- **Benefits**
 - Reduction of risk based on early feedback
 - Greater flexibility to accommodate new or changed requirements
 - Increased software quality



The Software Lifecycle

- The software lifecycle is broken into cycles where the output of a cycle is a generation of a product
- Each cycle is a succession of phases





a) Inception Phase

- This phase is concerned with establishing the scope of the project.
- **Purpose**
 - To establish the business case for a new system or for a major update of an existing system



a) Inception Phase

- **For small project**, this phase could be a simple chat over coffee and agreement to proceed.
- **For large project**, possible deliverable are:
 - A vision Document
 - An initial exploration of the customer's requirements.
 - An initial risk assessment.
 - Creating a Business Problem
 - Defining the scope of the project to understand the problem
 - A project plan.



b) Elaboration Phase

- **Purpose**

- Analyzing the problem domain
- Establishing a sound architectural foundation
- Addressing the highest risk elements of the project
- Creating a high-level analysis.
- Creating a construction plan that supports the goal of the project.

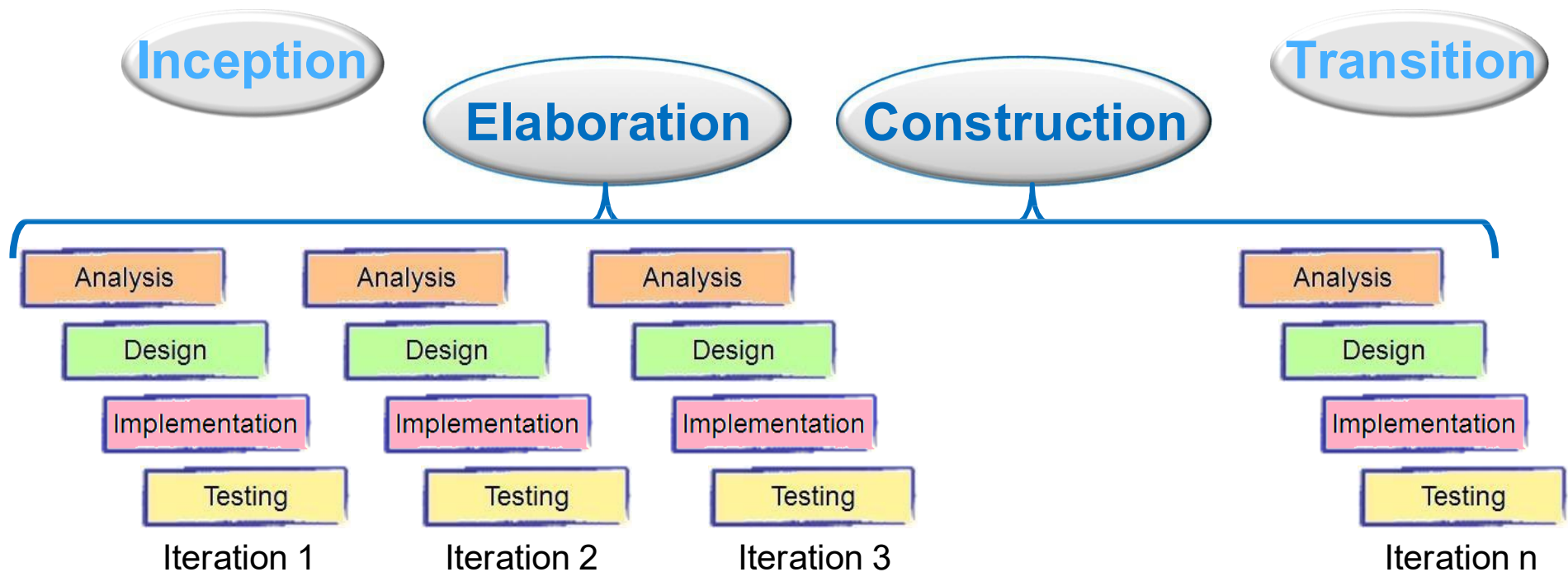


b) Elaboration Phase

- **Products**
 - An executable architecture
 - A baseline product vision based on the domain model
 - A revised risk assessment
 - A development plan
 - Evaluation criteria
 - Release descriptions

c) Construction Phase

- In this phase we build the product.
- In this phase, the product is build in the same fashion as the spiral model.





c) Construction Phase

- **Products**
 - A stream of executable releases
 - Quality assurance results
 - System and user documentation



d) Transition Phase

- The typical activities in this phase:
 - Introducing the release to the customer
 - Completing beta testing
 - Completing performance tuning, user training, and acceptance testing

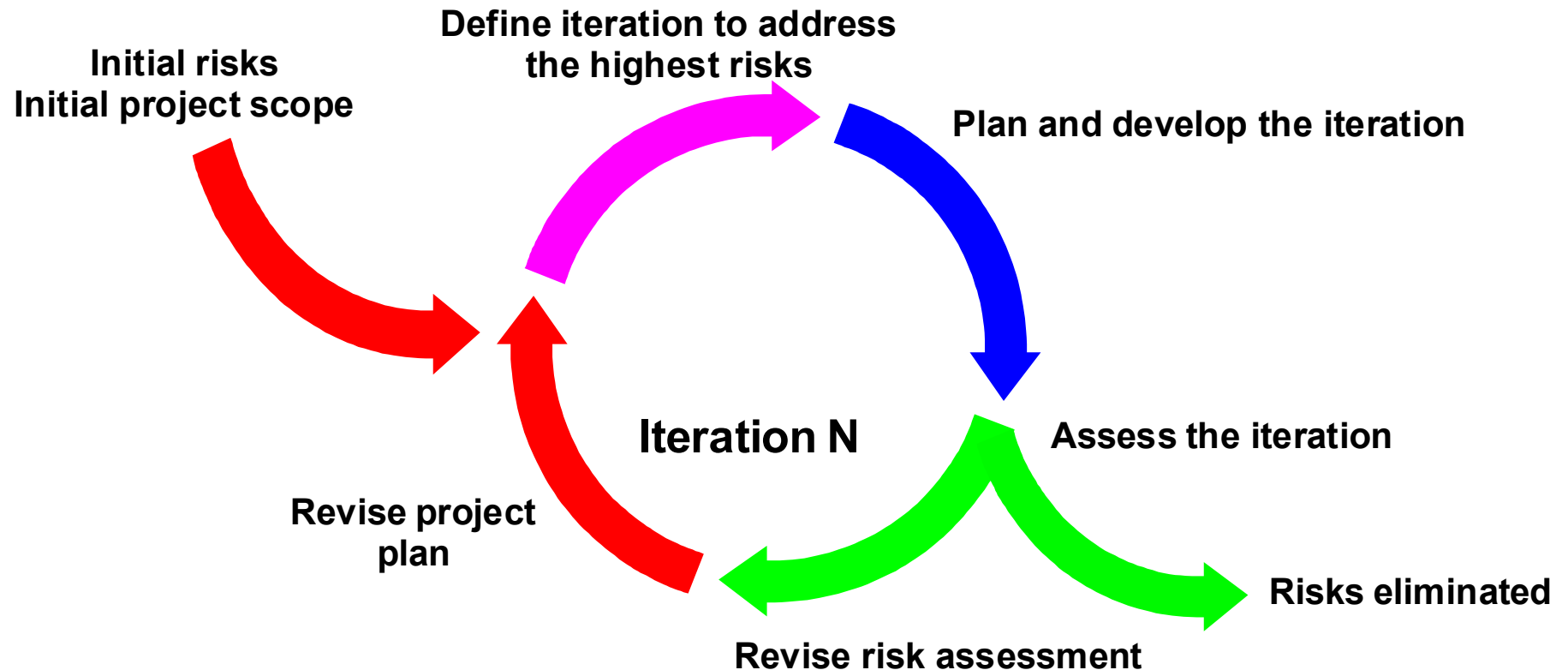


d) Transition Phase

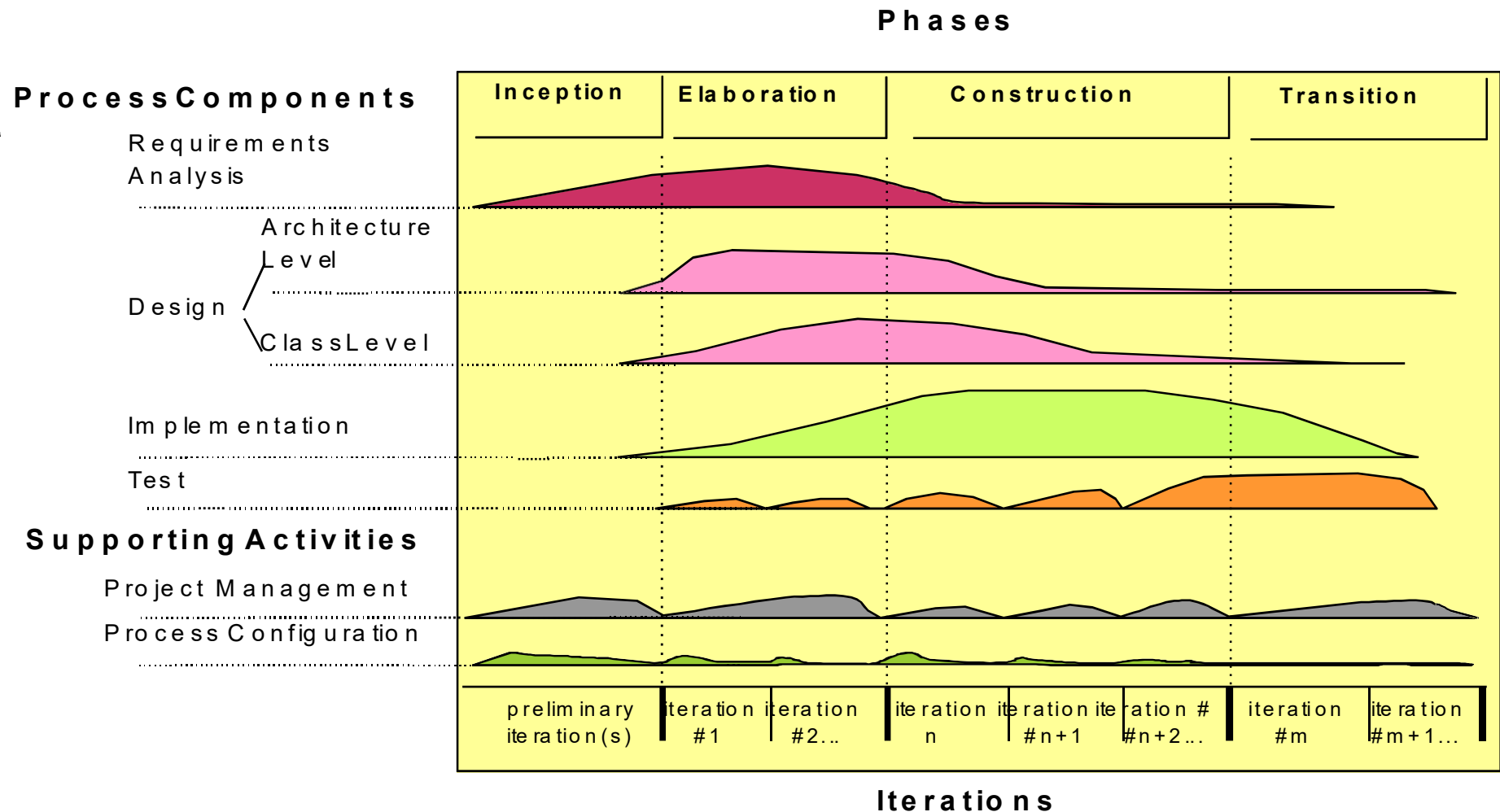
- **Products**
 - A stream of executable releases
 - Quality assurance results
 - Updated system and user documentation
 - “Investigation” analysis of project performance



Risk Reduction Drives Iterations



Putting It All Together

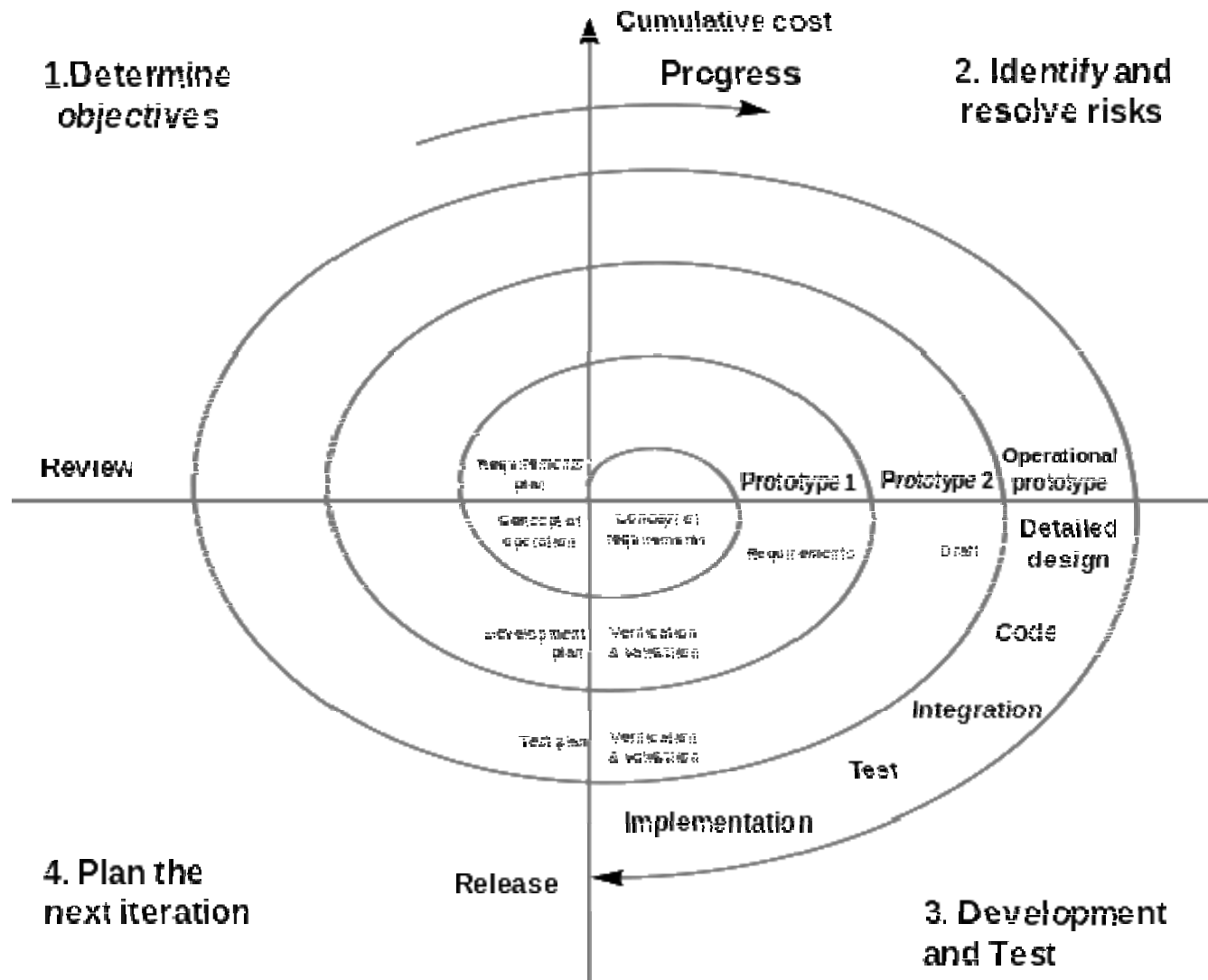




3. Spiral model

- The spiral model combines the idea of
 - iterative development with
 - the waterfall model.with very high emphasis on risk analysis.
- It allows for incremental releases of the product, or incremental refinement through each iteration around the spiral.

3. Spiral model





a) Identification

- This phase starts with gathering the business requirements in the baseline spiral.
- In the subsequent spirals,
 - identification of system requirements,
 - subsystem requirements and
 - unit requirements are all done in this phase.
- This also includes understanding the system requirements by continuous communication between the customer and the system analyst.
- At the end of the spiral the product is deployed in the identified market.



b) Design

- Design phase starts with the conceptual design in the baseline spiral.
- And involves architectural design, logical design of modules, physical product design and final design in the subsequent spirals.



c) Construct or Build

- Construct phase refers to production of the actual software product at every spiral.
- In the baseline spiral when the product is just thought of and the design is being developed a POC (Proof of Concept) is developed in this phase to get customer feedback.
- Then in the subsequent spirals with higher clarity on requirements and design details a working model of the software called build is produced with a version number. These builds are sent to customer for feedback.



d) Evaluation and Risk Analysis

- Risk Analysis includes identifying, estimating, and monitoring technical feasibility and management risks, such as schedule slippage and cost overrun.
- After testing the build, at the end of first iteration, the customer evaluates the software and provides feedback.

3. Spiral model

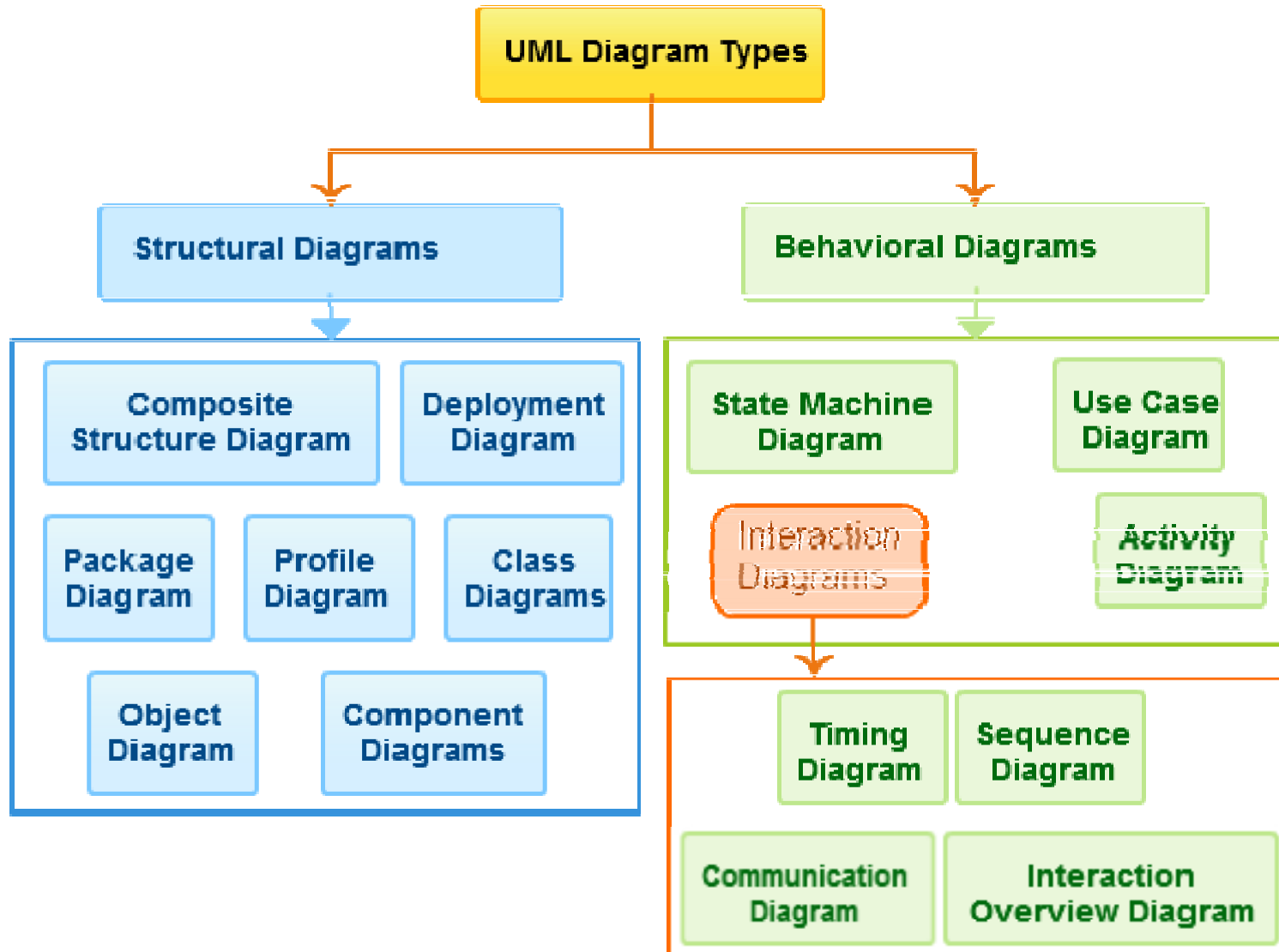
Pros	Cons
• Changing requirements can be accommodated. • Allows for extensive use of prototypes • Requirements can be captured more accurately. • Users see the system early. • Development can be divided into smaller parts and more risky parts can be developed earlier which helps better risk management.	• Management is more complex. • End of project may not be known early. • Not suitable for small or low risk projects and could be expensive for small projects. • Process is complex • Large number of intermediate stages requires excessive documentation.



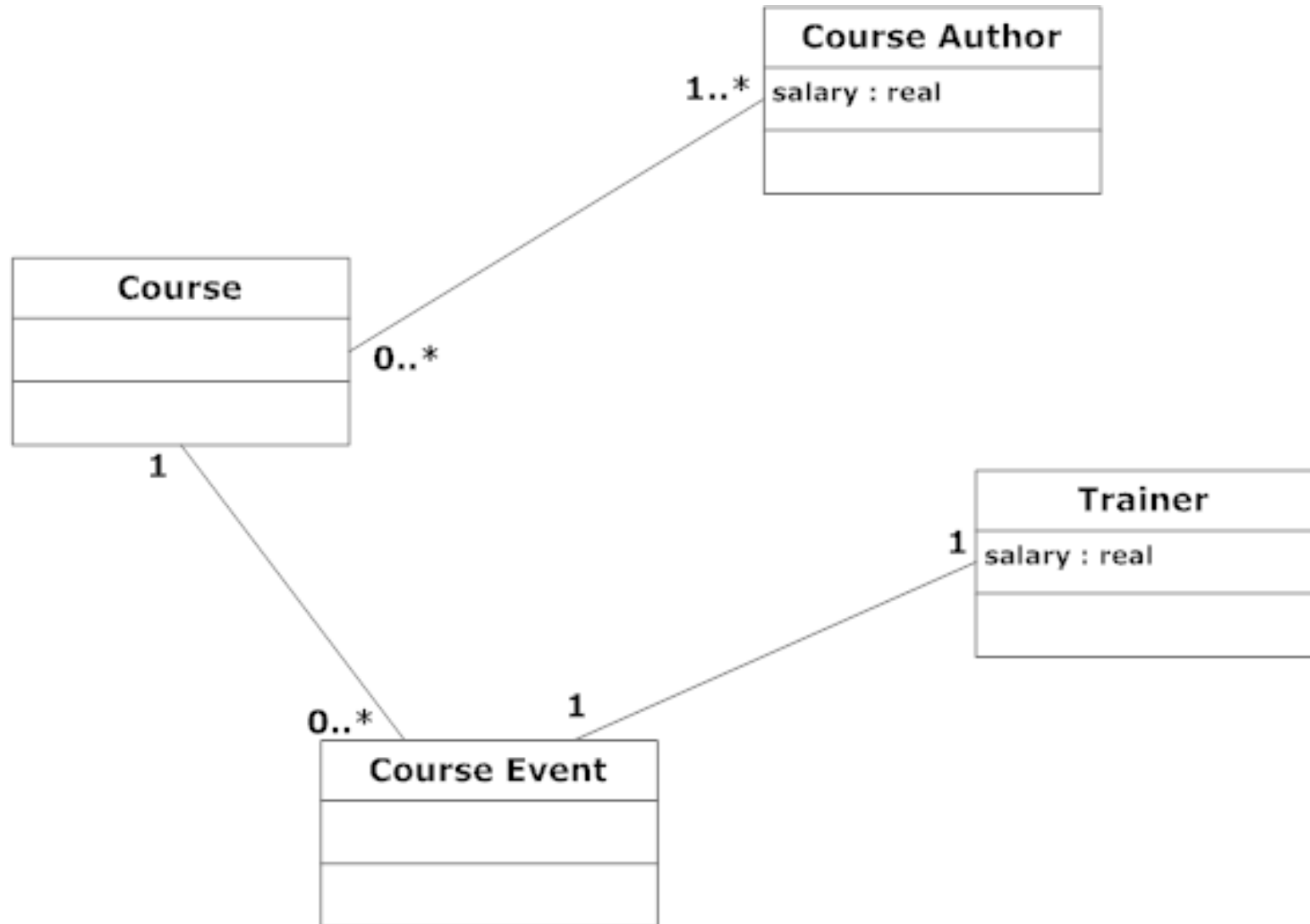
More Models

- V-Model
- Agile Model
- RAD Model
- Software Prototype Model

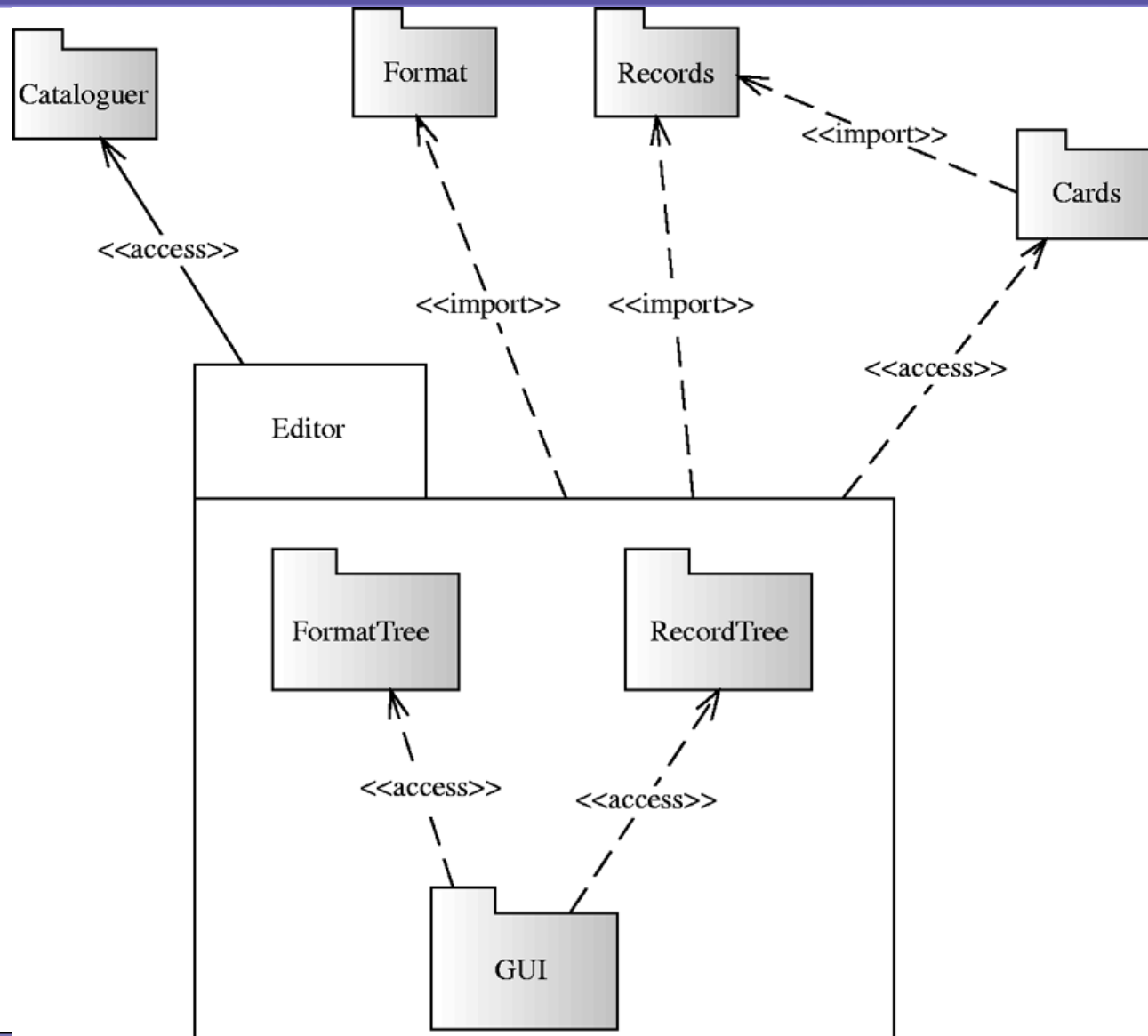
UML Diagram types



Class Diagram

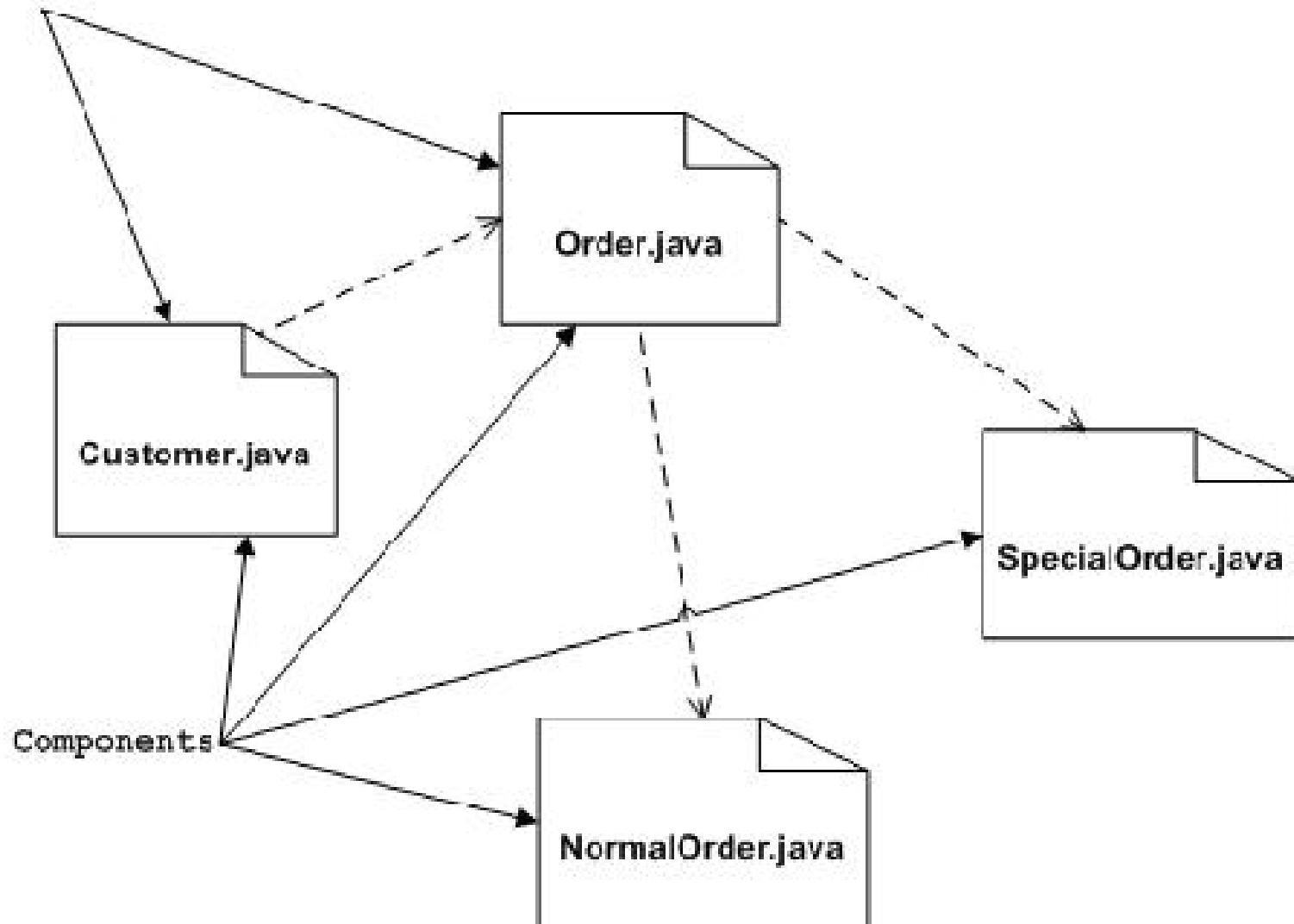


Package Diagram

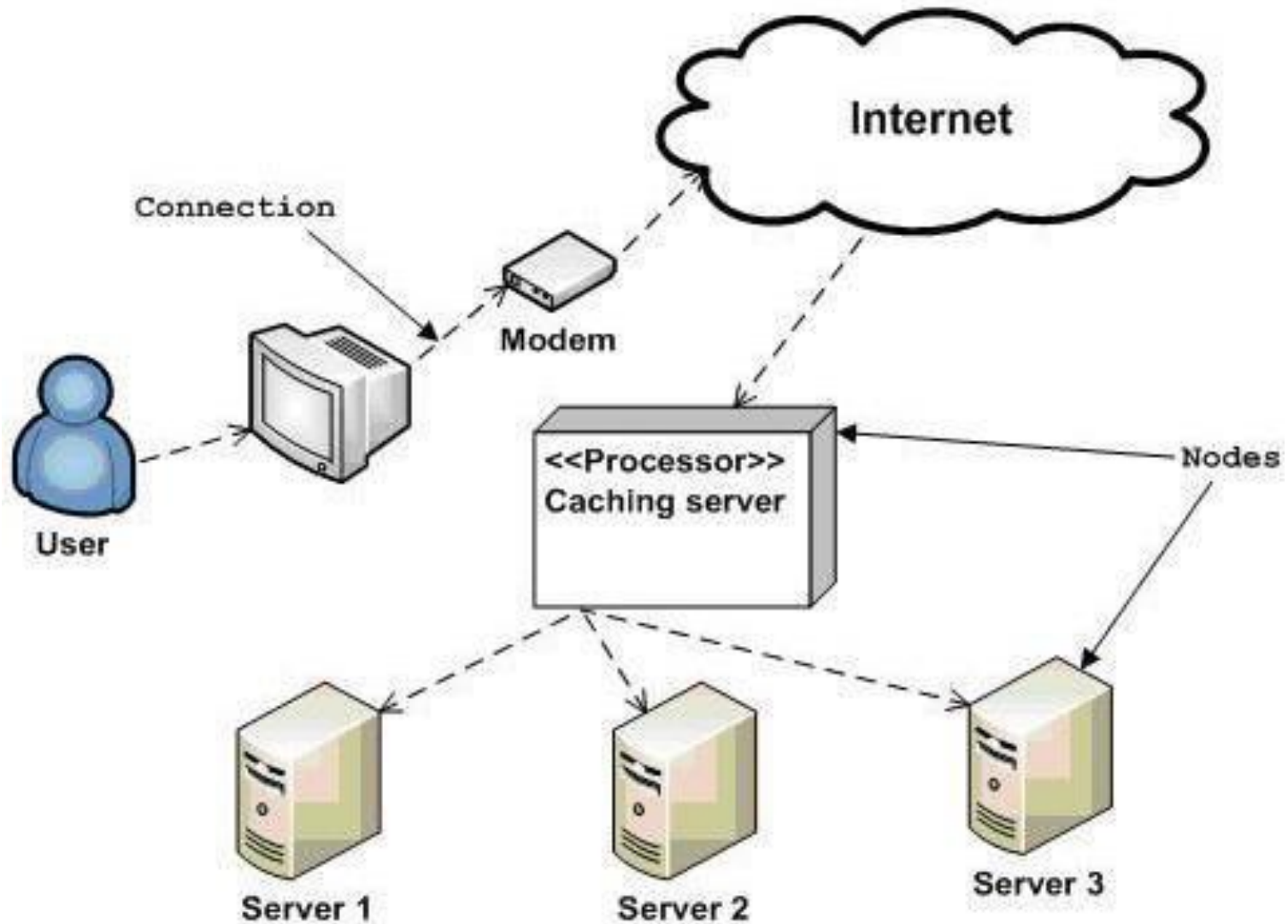


Component Diagram

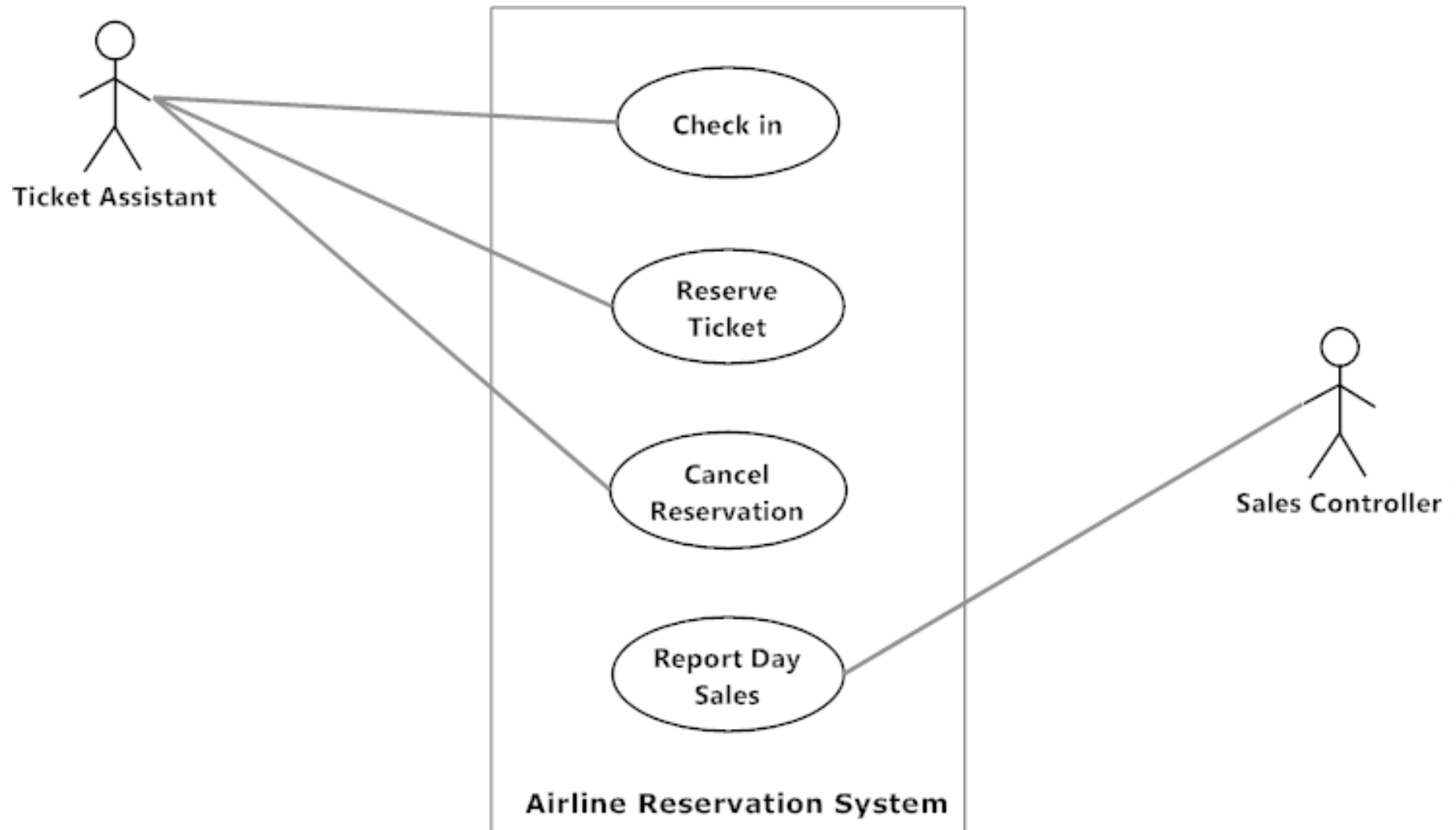
Java files



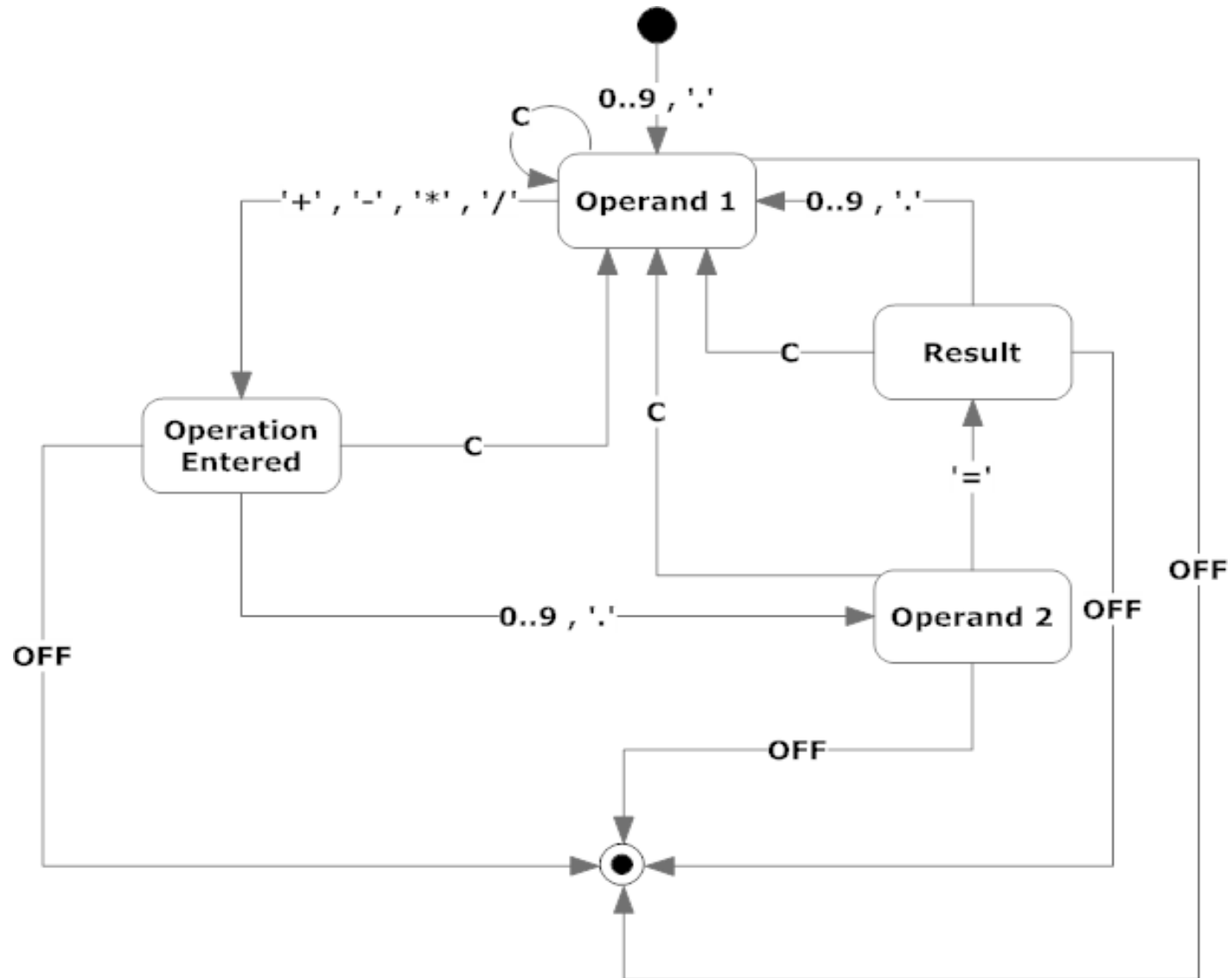
Deployment Diagram



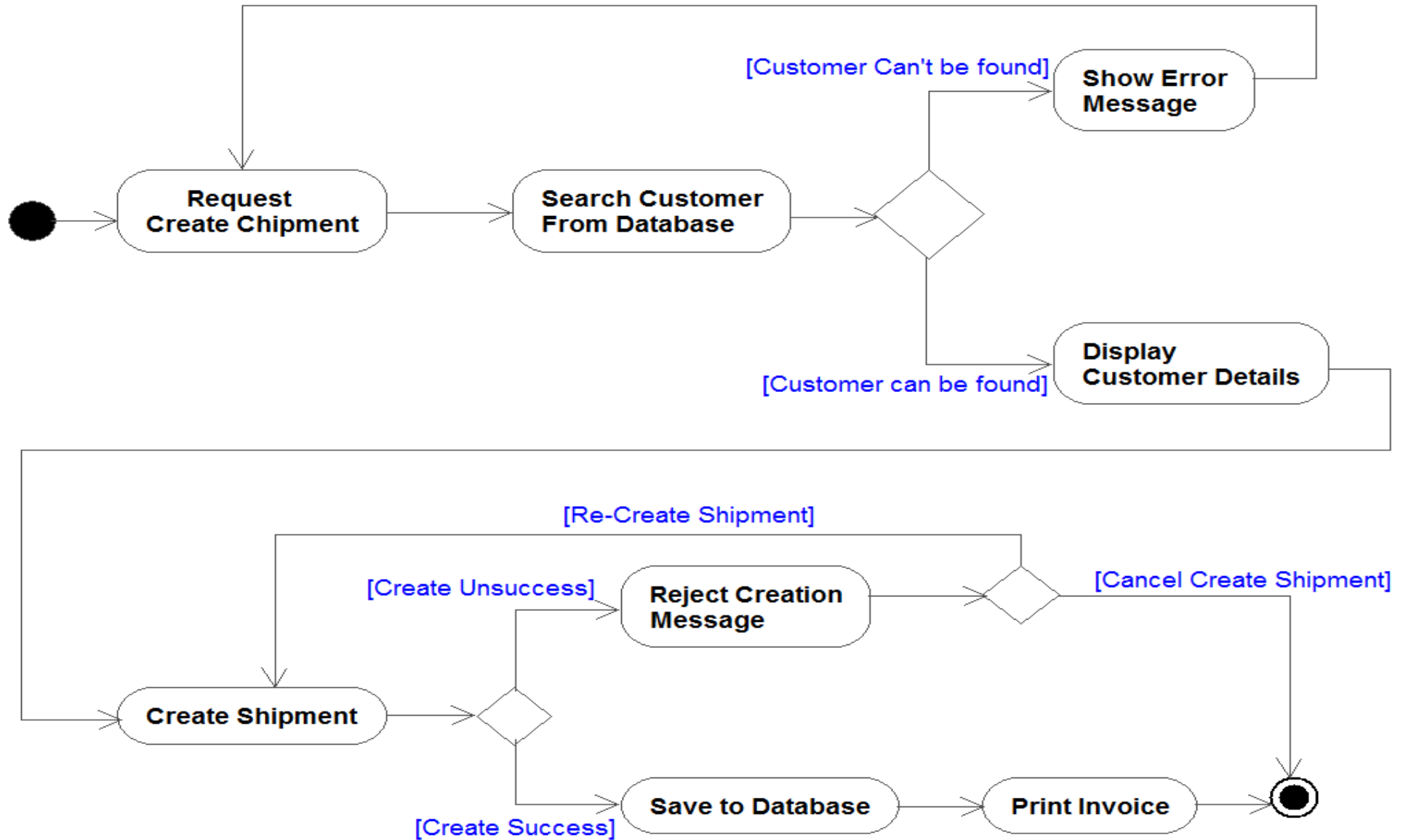
Use Case Diagram



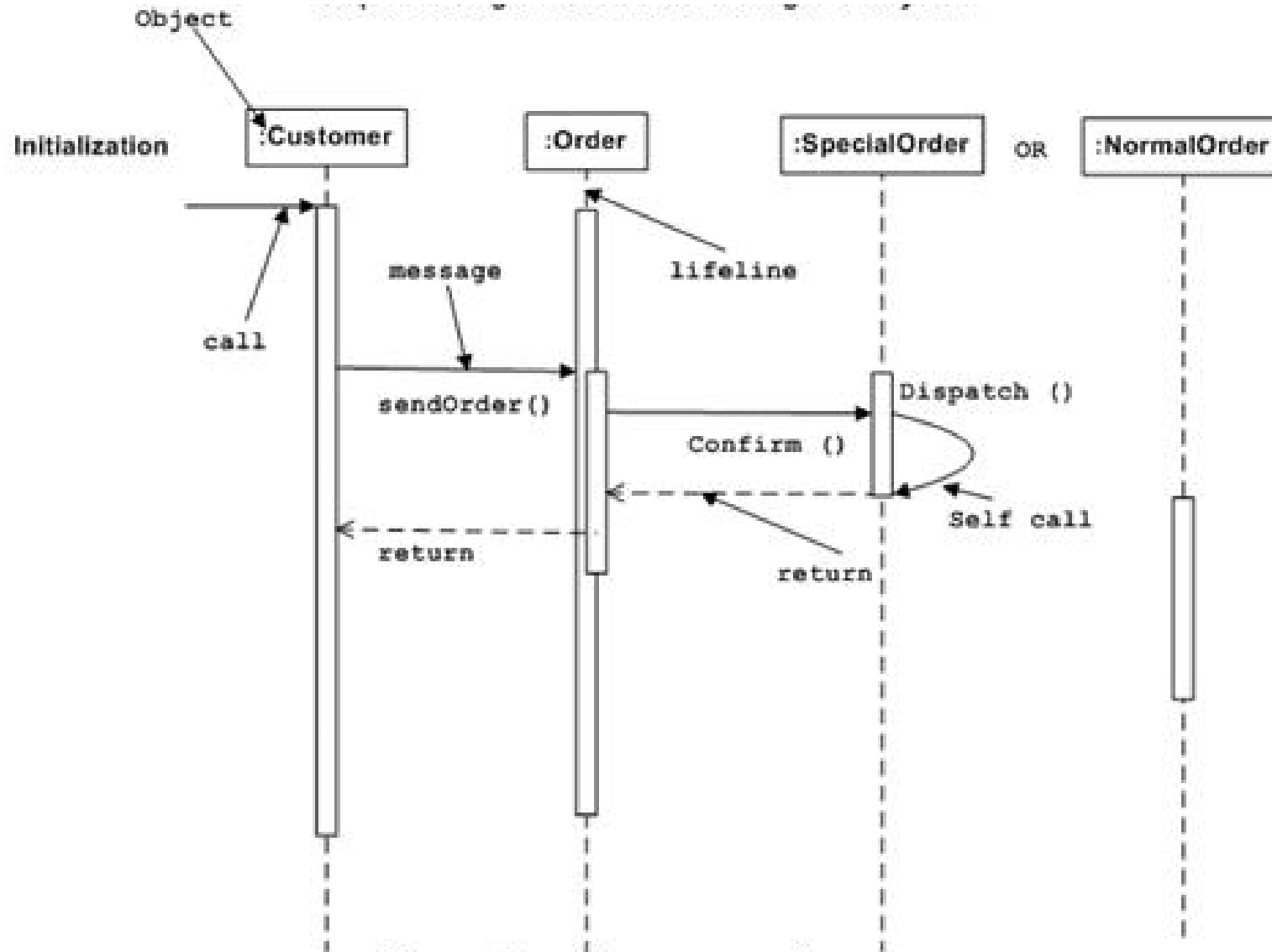
State Diagram



Activity Diagram



Sequence Diagram

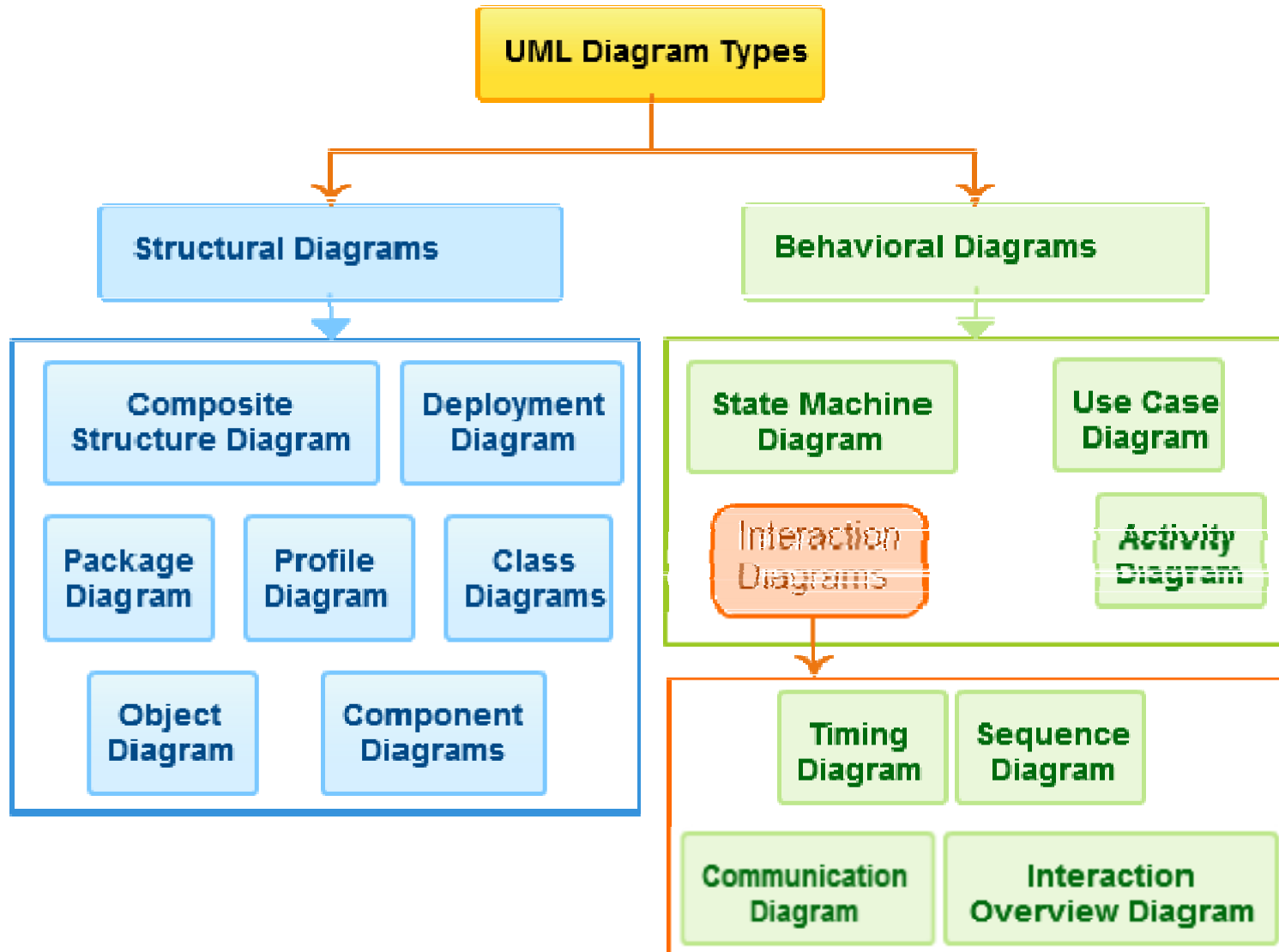




System Behavior

“use case diagram”

UML Diagram types





What is System Behavior ?

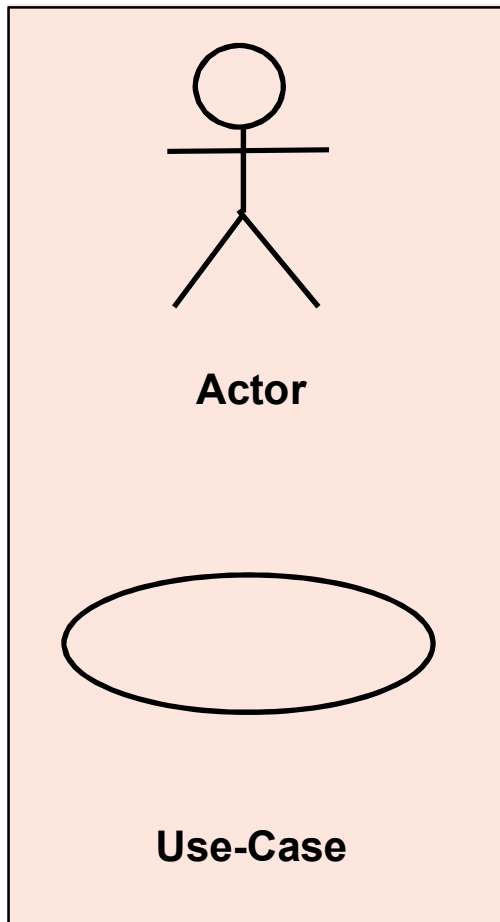
- **System behavior** is how a system acts and reacts
 - The visible and testable activity of a system
- System behavior is captured in use cases
 - They describe the system, its environment, and the relationship between the system and its environment



Benefits of a Use-Case Model

- **The use case model**
 - Is used to communicate with the end users and domain experts
 - Insures a mutual understanding of the requirements
 - Is used to identify
 - Who will interact with the system and what the system should do
 - Is used to verify
 - All requirements are captured
 - That the developers have understood the requirements

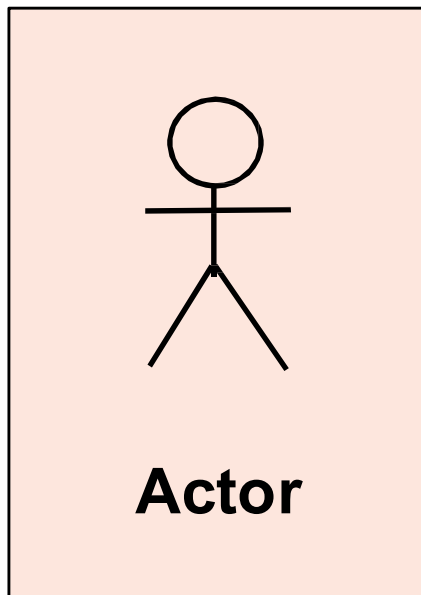
Major Concepts in Use-Case Modeling



- An actor
- A use case
- The use case model

What Is an Actor?

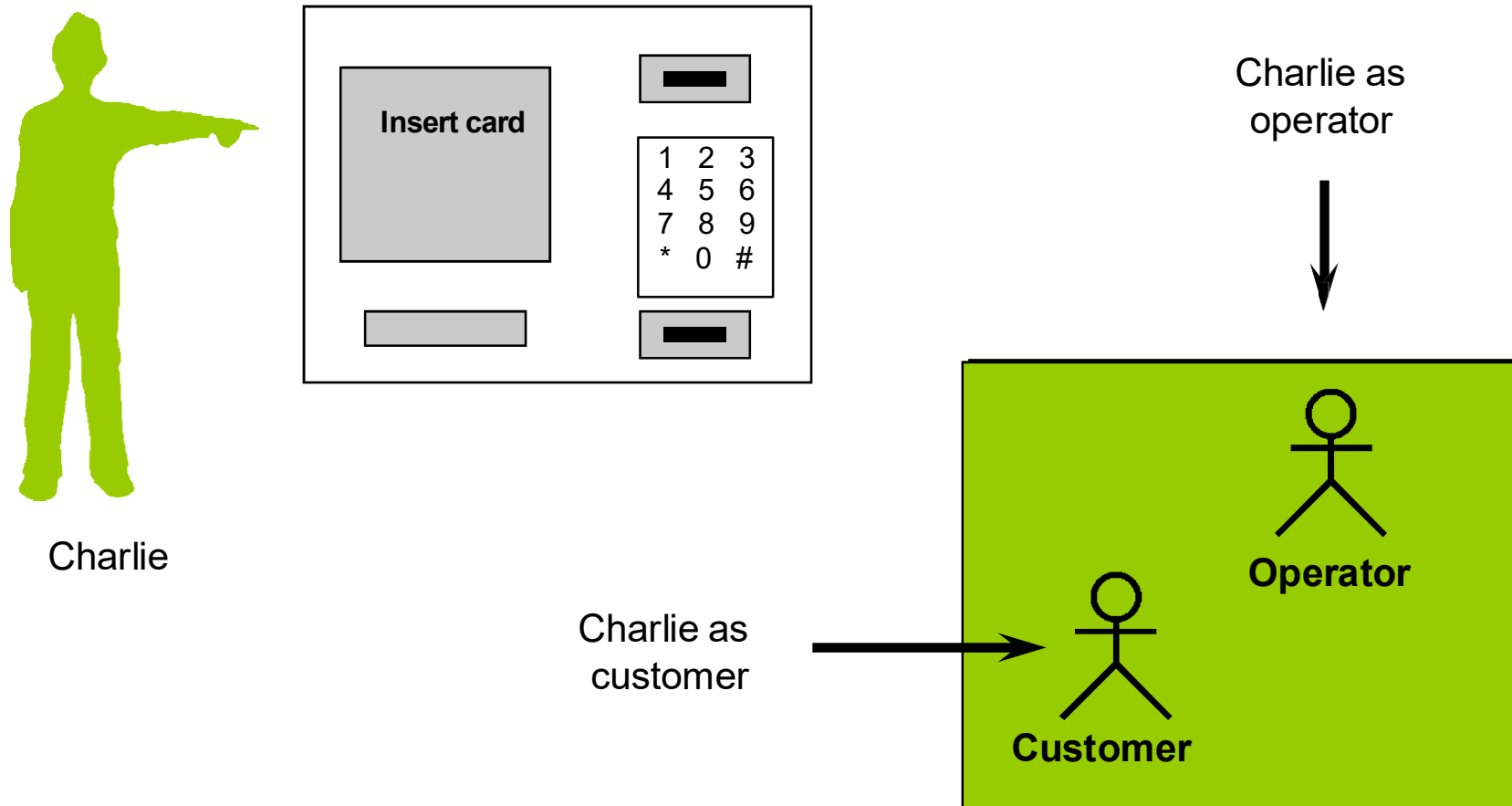
- **Actors**



- Represents anything that interacts with the system.
- Are not/or part of the system.
- Represents role of the system can play
- May actively interchange information with the system
- Can represent a human, a machine or another system

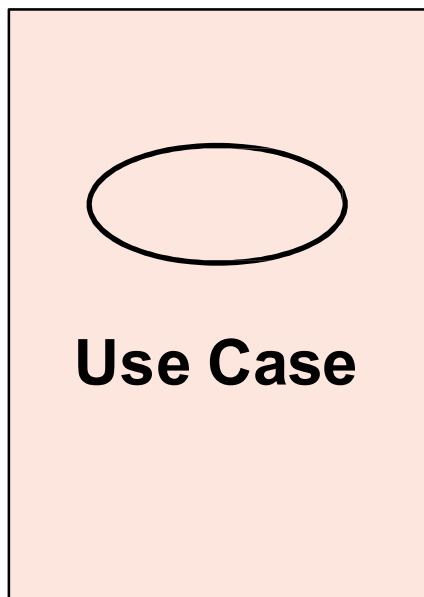


A User May Act as Several Actors



What Is a Use Case?

- **A use case**



- Is a case where your system is used to fulfill one or more of your user's requirements.
- Is initiated by an actor to invoke a certain functionality in the system
- Is a complete and meaningful flow of events



Sources of Info. for Use Cases

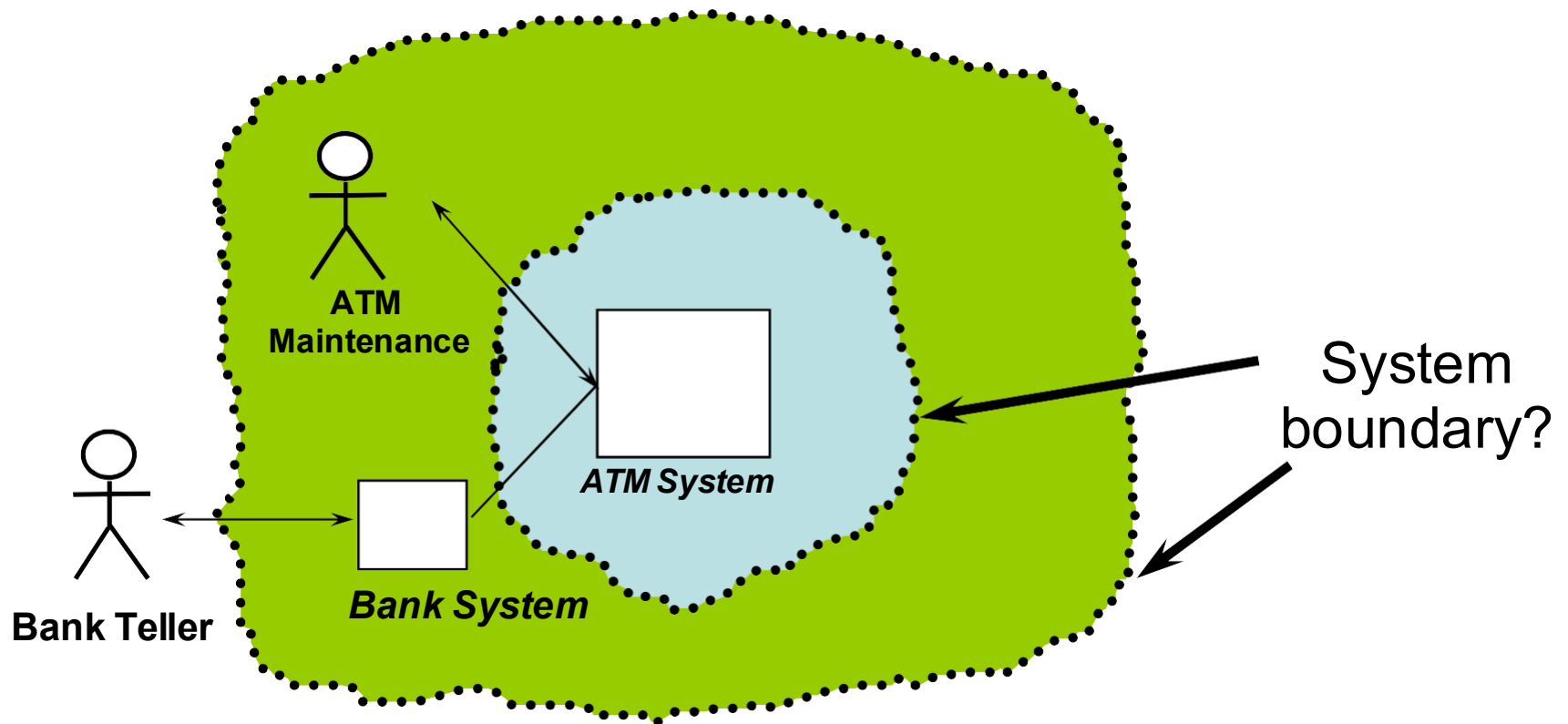
- System specifications/problem statement
- Interviews with domain experts
- Personal knowledge of the domain
- Legacy systems



What is a Use-Case Model ?

- **A use-case model:**
 - A model of the system's intended functions (use cases) and its surroundings (actors)
- **A use case diagram** is drawn to illustrate that use cases and actors interact by sending stimuli to one another

Actors and System Boundaries



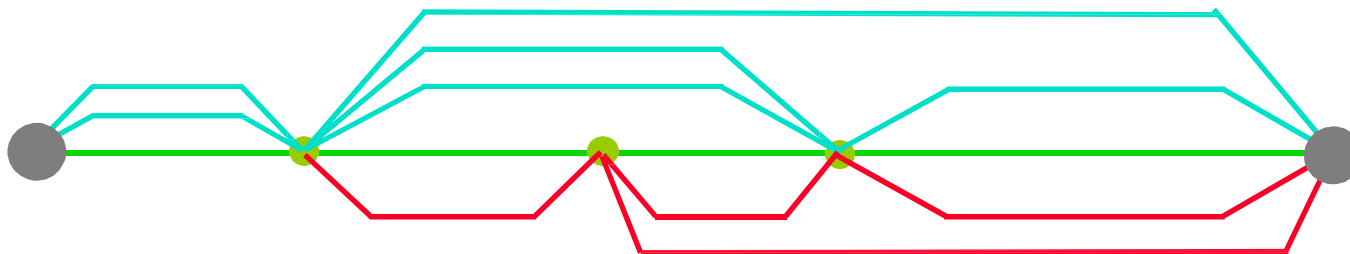


Use Case Documentation

- Use cases are documented in
 - A brief description
 - The purpose of the use case in a few lines
 - Detailed flow of events
 - Description of the primary and alternate flow of events that occur when the use case is initiated
- Both documents are written in terms the customer will understand

Use Case Flow of Events

- Each use case
 - Has one normal, basic, sequence of transactions
 - May have several alternative sequences of transactions
 - Usually has several exceptional sequences of transactions handling erroneous situations
 - May also have well defined pre- and post-conditions





Use Case Flow of Events

- Describe only the events that belong to the use case, and not what happens in other use cases
- Avoid vague terminology such as "for example", "etc." and "information"



Use Case Flow of Events

- The flow of events should describe
 - How and when the use case starts and ends
 - When the use case interacts with the actors
 - What information is exchanged between an actor and the use case
 - Do not describe the details of the user interface, unless it is an important requirement
 - The basic flow of events
 - Any alternate flow of events

Who Reads Use-Case Doc.?



- **Customer:** approve what the system should do
- **Users:** provide system understanding
- **Developers:** document system behavior
- **Reviewers:** Examine the flow of events
- **Analysts (Designer):** provide the basis for analysis and design
- **Tester:** as a base for test cases
- **Project Leader:** project planning
- **Technical Writer:** When writing the user's guide for the end users



Course Registration Example





Course Registration Example

- At the beginning of each semester **students** may **request a course catalogue** containing a list of course offerings for the semester.
- Information about each course, such as **professor**, **department**, and **prerequisites** will be included to help students make informed decisions.
- The new system will allow students to **select four course offerings** for the coming semester.
- In addition, each student will **indicate two alternative choices** in case a course offering becomes filled or canceled.



Course Registration Example

- No course offering will have **more than ten students**.
- No course offering will have **fewer than three students**.
 - A course offering with fewer than three students will be canceled.
- Once the registration process is completed for a student, the registration system sends information to **the billing system** so the student can be billed for the semester.



Course Registration Example

- **Professors** must be able to access the on-line system to **indicate which courses they will be teaching**.
- They will also need to **see which students signed up** for their course offerings.
- For each semester, there is a period of time that **students can change their schedule**.
- Students must be able to access the system during this time to **add** or **drop** courses.



Actors in the Course Registration Ex.

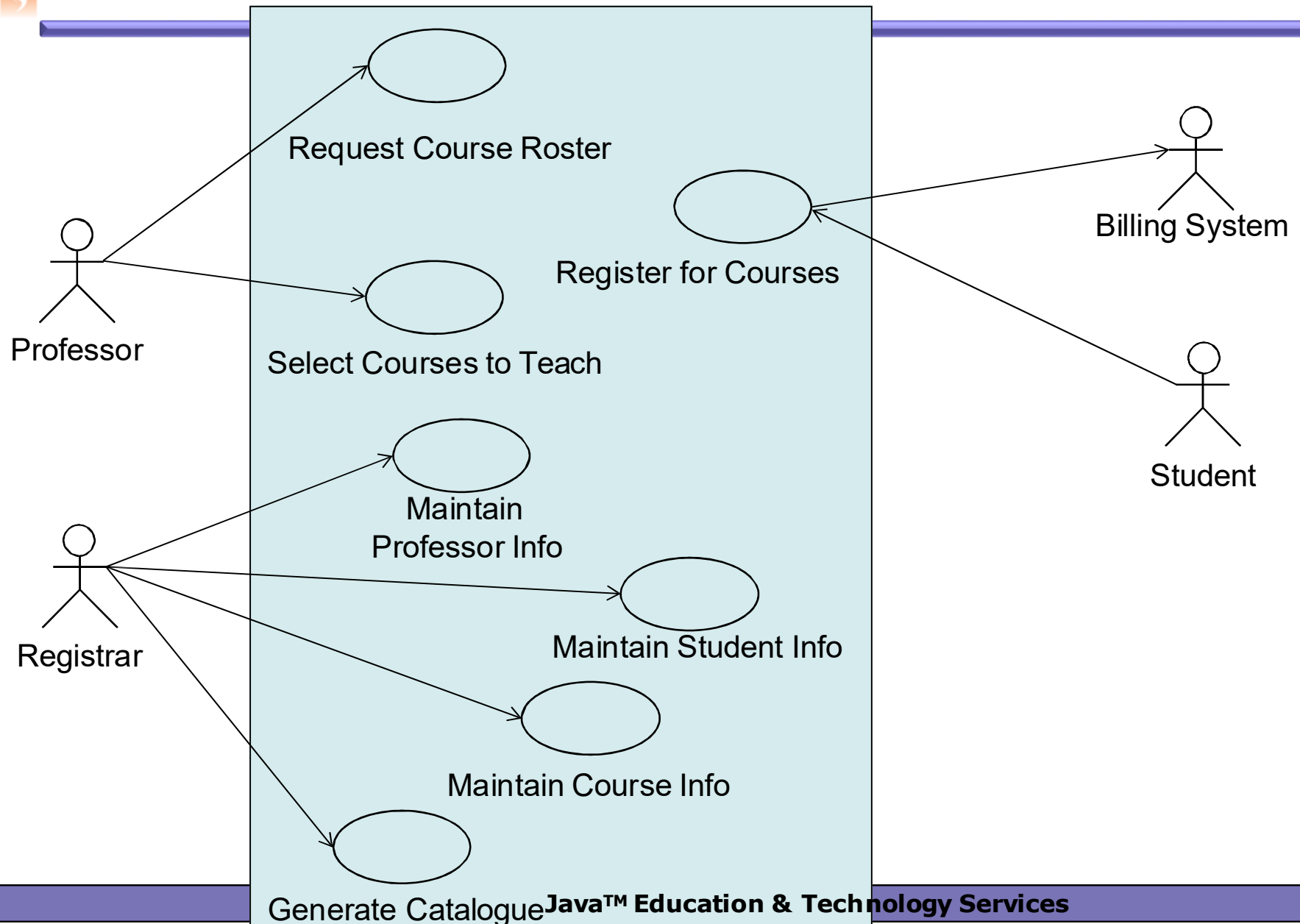
- The actors are
 - Student
 - Professor
 - Billing System
 - **Registrar ????????????????**



Actors

- Student
 - Register for courses
- Registrar
 - Maintain course information, Maintain student information, Maintain professor information, Generate catalogue
- Professor
 - Request course roster, Select courses to teach
- Billing System
 - Receive info. from registration process.

Use Case Diagram





Brief Description, Register for Courses

This use case is initiated by a student. It provides the capability for the student to create, delete, modify and/or review a course schedule for a given semester.



Flow of Events, Register for Courses

1. Student enters the student id number.
2. The system verifies that the student id number is valid and prompts the student to select the semester.
3. The student enters the desired semester.
4. The system prompts the student to select the desired activity:
 - a) Create a schedule
 - b) Review a schedule
 - c) Change a schedule: Delete a course , Add a course



Flow of Events, Register for Courses

5. The student indicates that the activity is complete.
6. The system will print the student schedule and notify the student that registration is complete.
7. The system sends billing information for the student to the billing system for processing.



Flow of Events, Register for Courses

- **Alternate flow:** If an invalid id number is entered, the system will not allow access to the registration system.
- **Alternate flow:** If an attempt is made to create a schedule for a semester where a schedule already exists the system will prompt for another choice to be made.



Flow of Events, Register for Courses

a) Create a schedule:

- The student enters 4 primary course offering numbers and 2 alternate course offering numbers.
- The system then
 - Checks that prerequisites are satisfied for the requested course
 - Adds the student to the course offering if the course offering is open.
- **Alternate flow:** If a primary course offering is not available, the system will substitute an alternate course offering.



Flow of Events, Register for Courses

- **Review a Schedule:**
 - The student requests information on all course offerings in which the student is registered for a given semester.
 - The system displays all course offerings for which the student is registered including
 - course name, course number, course offering number, days of the week, time, location, and number of credit hours.



Flow of Events, Register for Courses

- **Change Schedule:**

- Delete a Course Offering The student indicates which course offering to delete.
- The system checks that the final date for changes has not been exceeded.
- The system deletes the student from the course offering.
- The system notifies the student that the request has been processed.



Flow of Events, Register for Courses

- **Change Schedule, Add a Course**
 - The student indicates which course offerings to add.
 - The system checks that the final date for changes has not been exceeded.
 - The system then
 - Verifies that the maximum course load for the student has not been exceeded
 - Checks that prerequisites are satisfied for the requested course
 - Adds the student to the course offering if the course offering is open.



Structural Diagrams

Class Diagrams

A thick blue horizontal bar at the bottom of the slide.



UML class diagrams

What is a UML class diagram?

- **UML class diagram:** a picture of the classes in an OO system, their fields and methods, and connections between the classes that interact or inherit from each other

What are some things that are not represented in a UML class diagram?

- details of how the classes interact with each other
- algorithmic details; how a particular behavior is implemented



Diagram of one class

class name in top of box

write <<interface>> on top of interfaces' names
use *italics* for an *abstract class* name

attributes (optional)

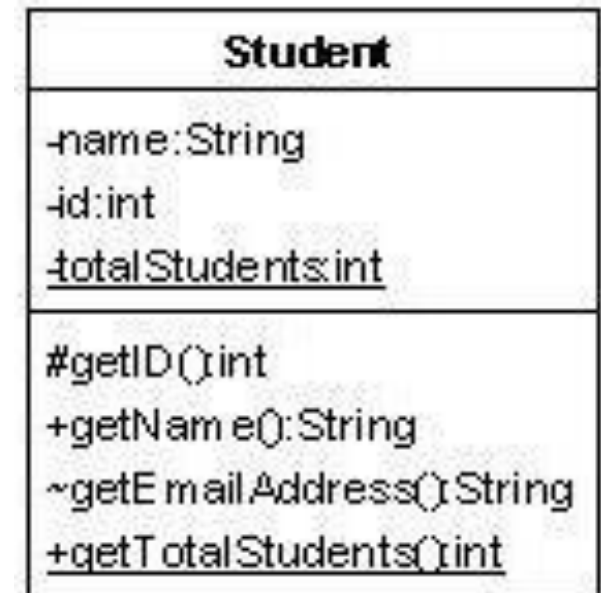
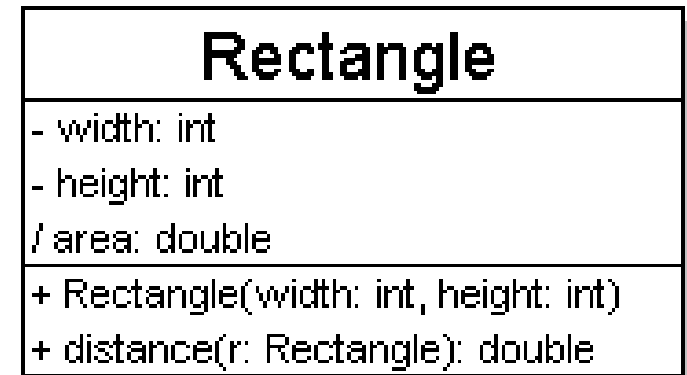
should include all fields of the object

operations / methods (optional)

may omit trivial (get/set) methods

but don't omit any methods from an interface!

should not include inherited methods





Class attributes

attributes (fields, instance variables)

visibility name : type [count] = default_value

visibility:	+	public
	#	protected
	-	private
	~	package (default)
	/	derived

underline static attributes

derived attribute: not stored, but can be computed from other attribute values

attribute example:

- balance : double = 0.00

Rectangle
- width: int - height: int / area: double
+ Rectangle(width: int, height: int) + distance(r: Rectangle): double

Student
-name:String -id:int <u>-totalStudents:int</u>
#getID():int +getName():String ~getEmailAdress():String <u>+getTotalStudents():int</u>



Class operations / methods

operations / methods

visibility name (parameters) : return_type

visibility: + public
 # protected
 - private
 ~ package (default)

underline static methods

parameter types listed as (name: type)

omit *return_type* on constructors and

when return type is void

method example:

+ distance(p1: Point, p2: Point): double

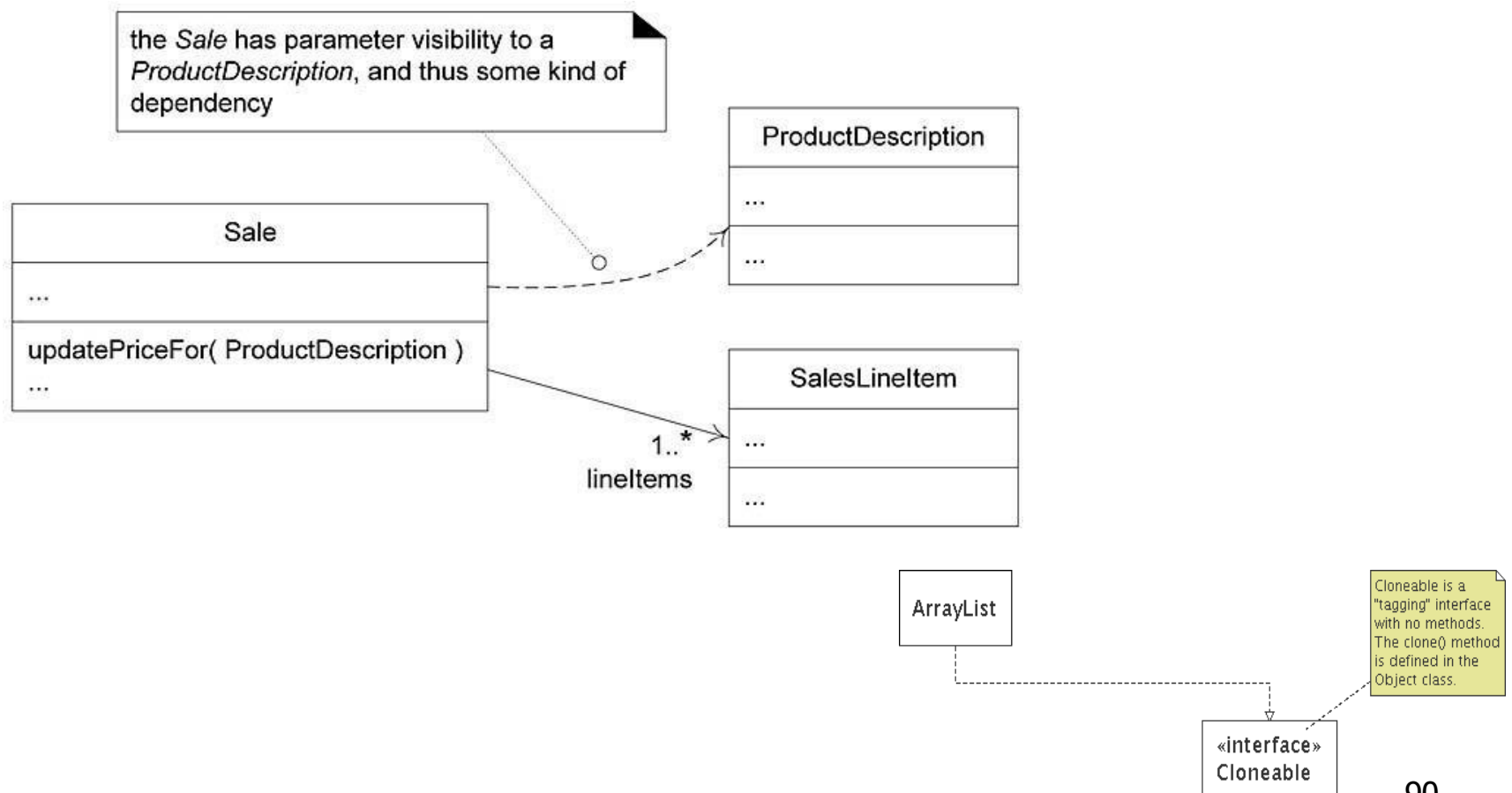
Rectangle
- width: int - height: int / area: double
+ Rectangle(width: int, height: int) + distance(r: Rectangle): double

Student
-name:String -id:int <u>-totalStudents:int</u>
#getID()int +getName():String ~getEmailAdress()String <u>+getTotalStudents()int</u>



Comments

represented as a folded note, attached to the appropriate class/method/etc by a dashed line





Relationships btween classes

generalization: an inheritance relationship

inheritance between classes

interface implementation

association: a usage relationship

dependency

aggregation

composition



Generalization relationships

generalization (inheritance) relationships

hierarchies drawn top-down with arrows pointing upward to parent

line/arrow styles differ, based on whether parent is a(n):

class:

solid line, black arrow

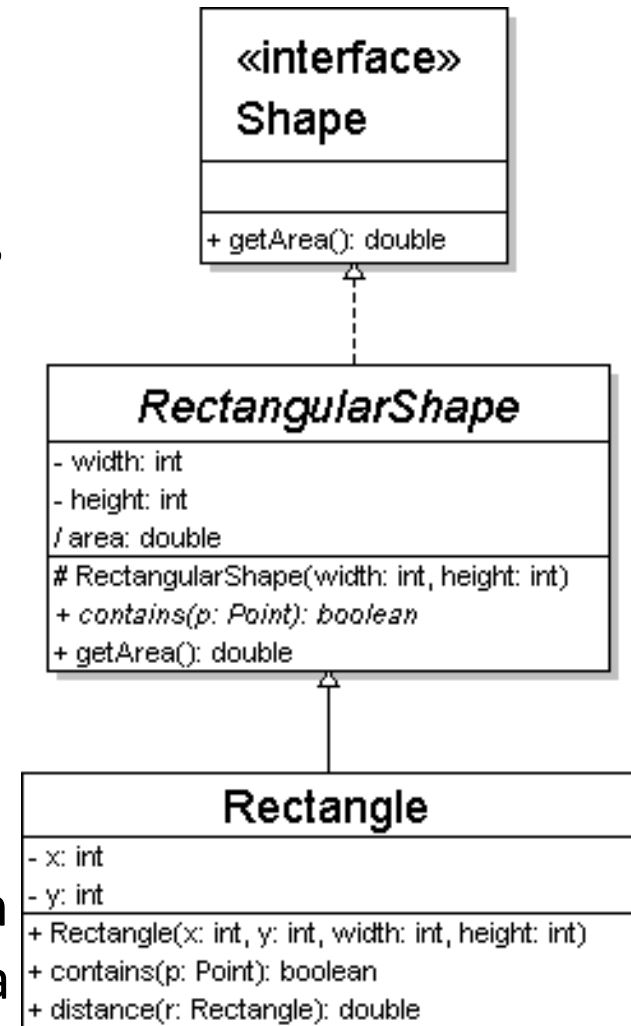
abstract class:

solid line, white arrow

interface:

dashed line, white arrow

we often don't draw trivial / obvious generalization relationships, such as drawing the Object class as a parent

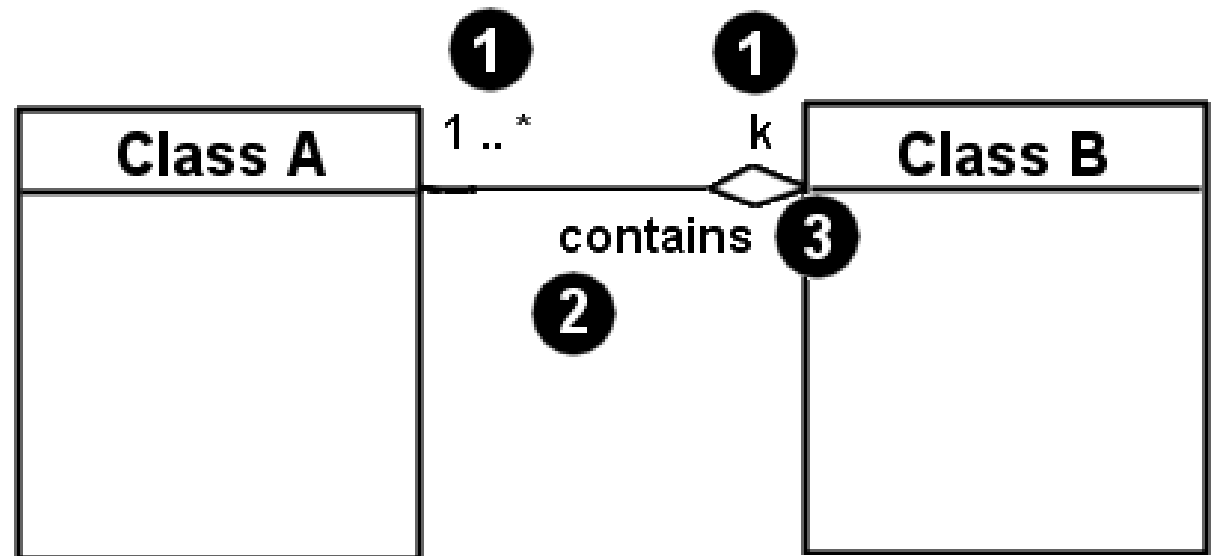




Associational relationships

associational (usage) relationships

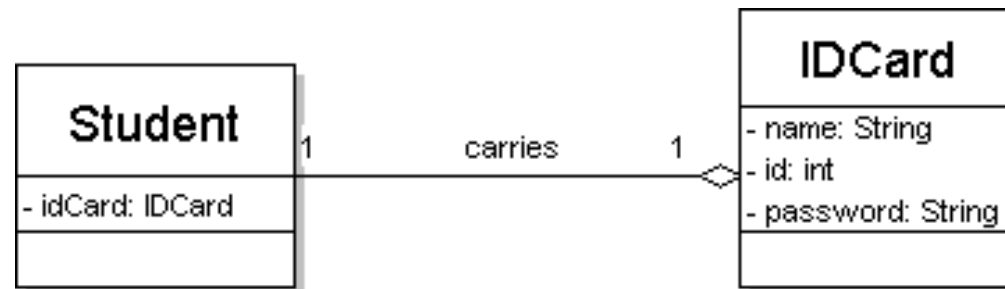
1. multiplicity (how many are used)
 - * $\Rightarrow$ 0, 1, or more
 - 1 $\Rightarrow$ 1 exactly
 - 2..4 $\Rightarrow$ between 2 and 4, inclusive
 - 3..* $\Rightarrow$ 3 or more
2. name (what relationship the objects have)
3. navigability (direction)



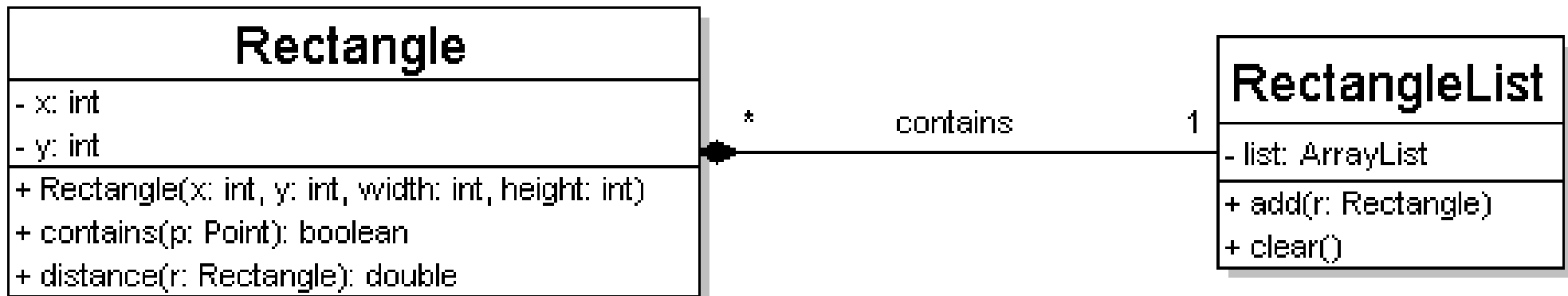


Multiplicity of associations

- one-to-one
 - each student must carry exactly one ID card



- one-to-many
 - one rectangle list can contain many rectangles





Association types

aggregation: "is part of"

symbolized by a clear white diamond

composition: "is entirely made of"

stronger version of aggregation

the parts live and die with the whole

symbolized by a black diamond

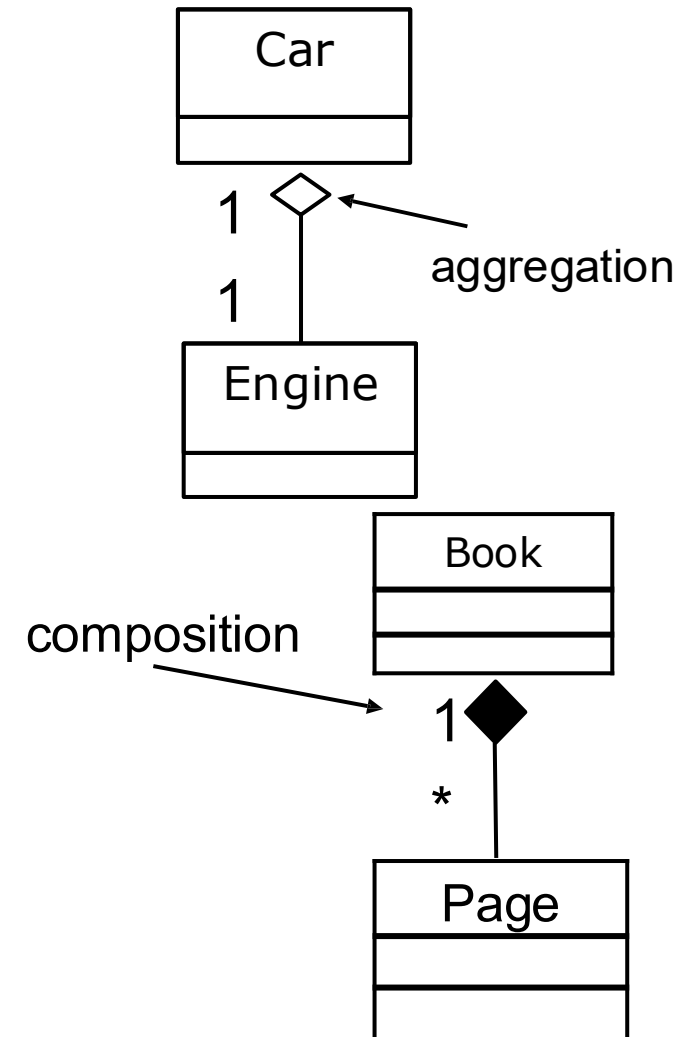
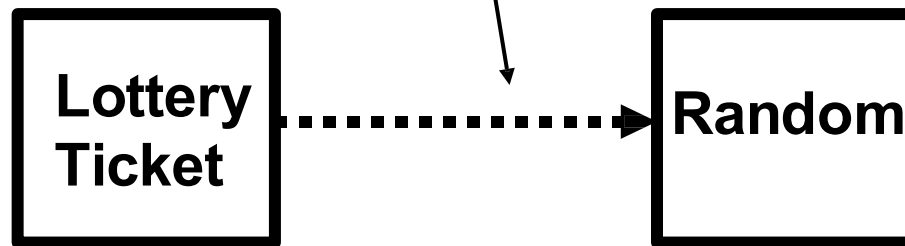
dependency: "uses temporarily"

symbolized by dotted line

often is an implementation

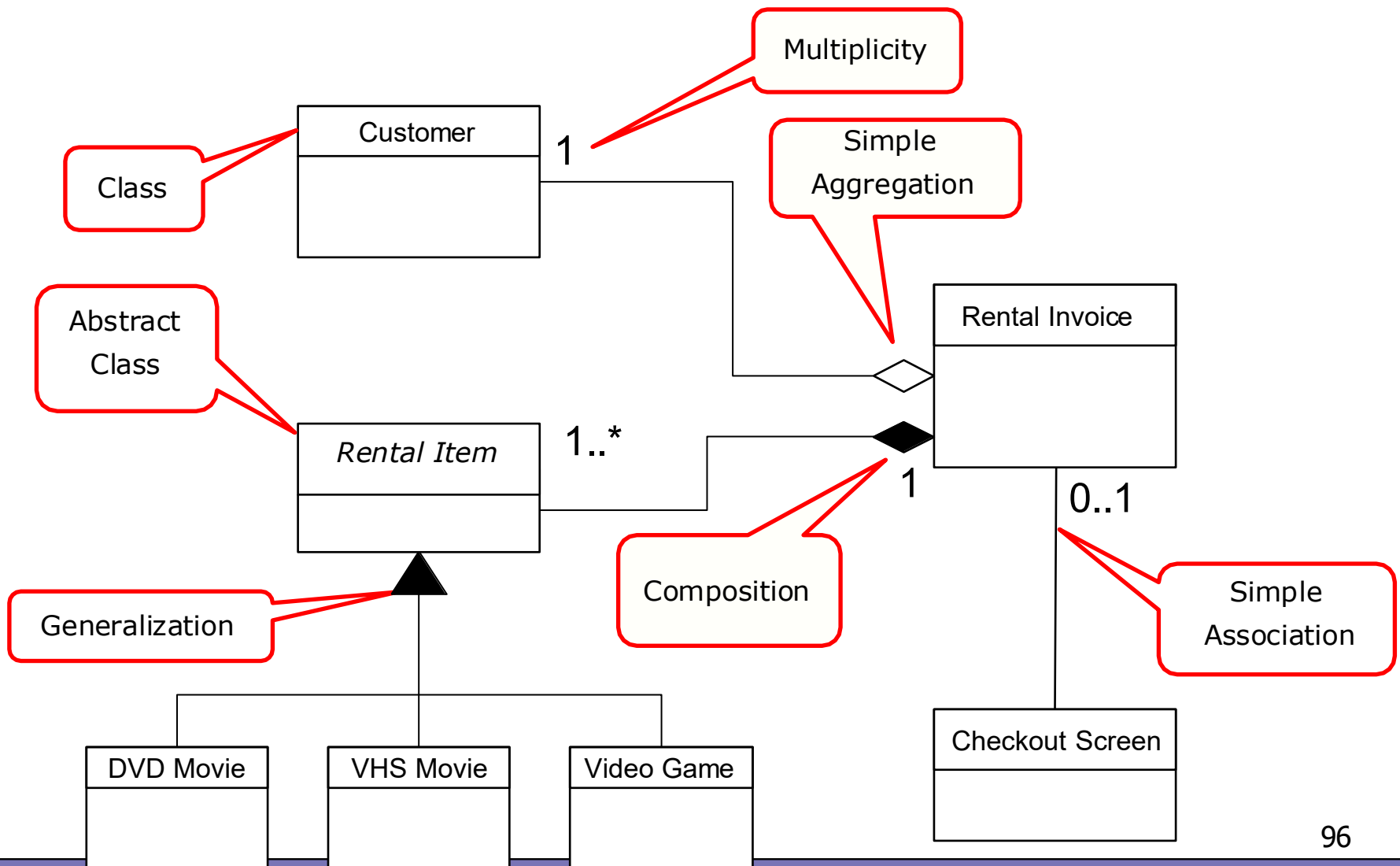
detail, not an intrinsic part of

that object's state



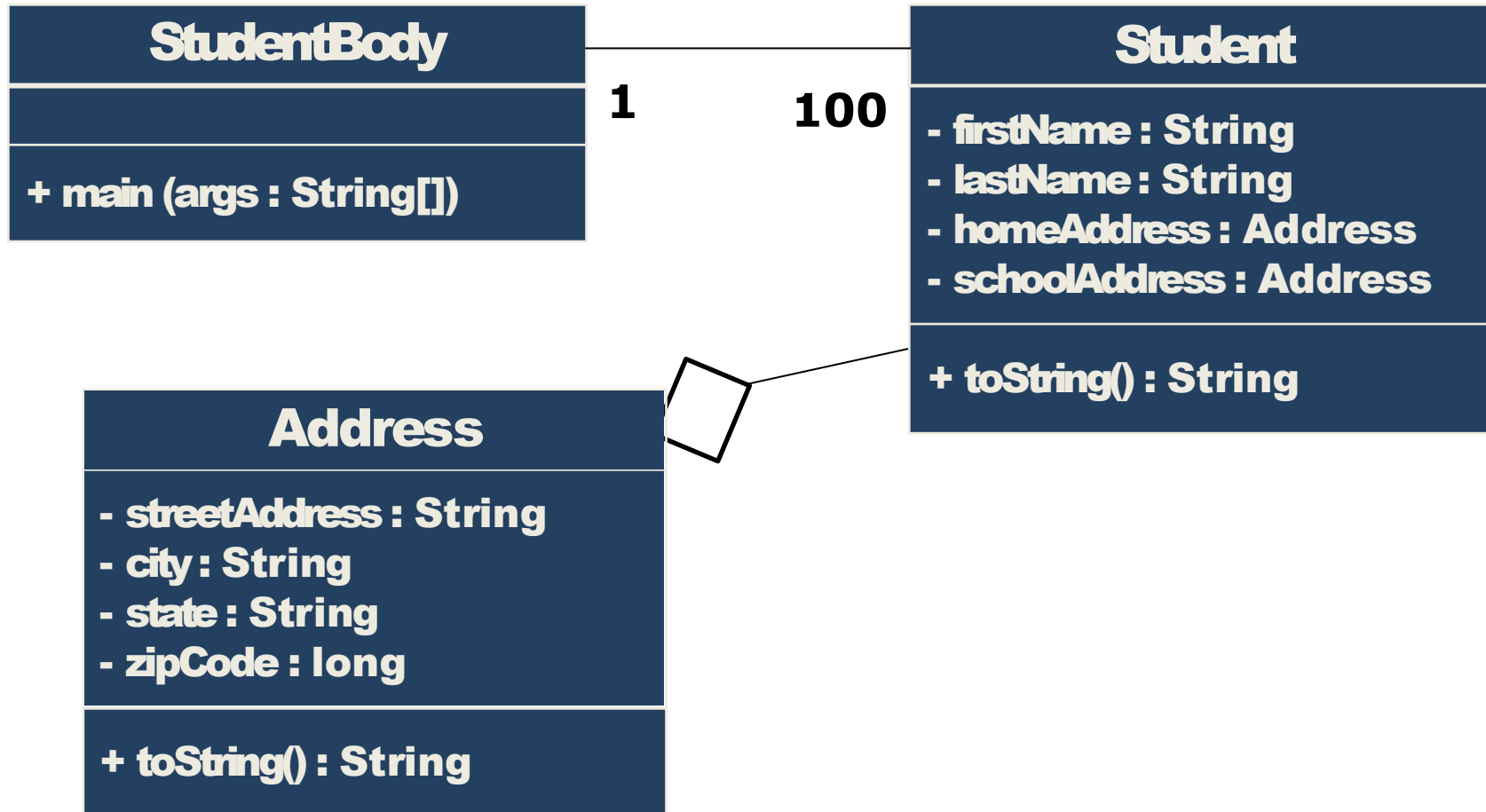


Class diagram example





Class diagram example 2





Tools for creating UML diagrams.

Violet (free)

<http://horstmann.com/violet/>

Rational Rose

<http://www.rational.com/>

Visual Paradigm UML Suite (trial)

<http://www.visual-paradigm.com/>

(nearly) direct download link:

<http://www.visual-paradigm.com/vp/download.jsp?product=vpuml&edition=ce>

(there are many others, but most are commercial)

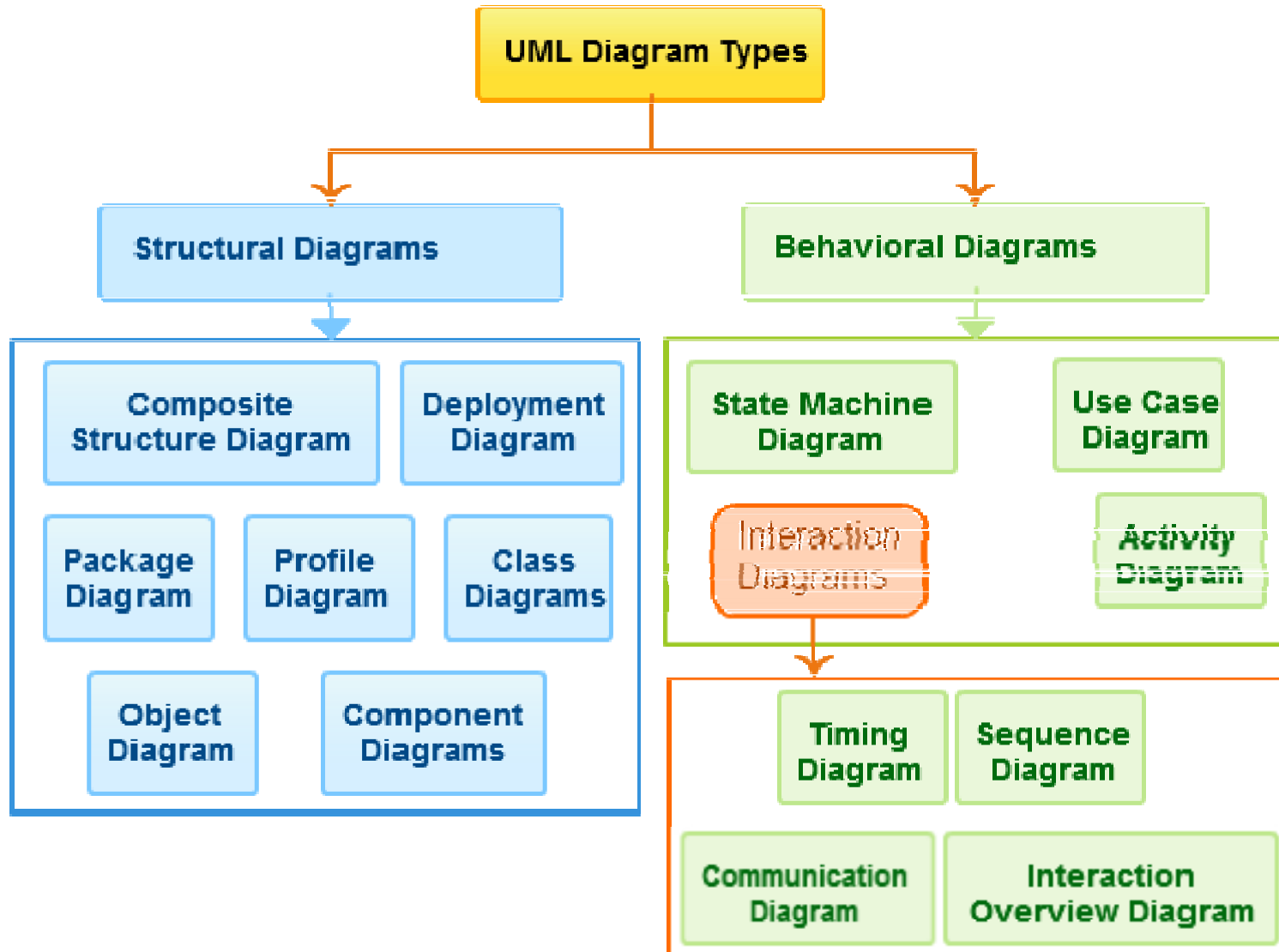


Class design exercise



Object Interaction

UML Diagram types





Interaction Diagrams

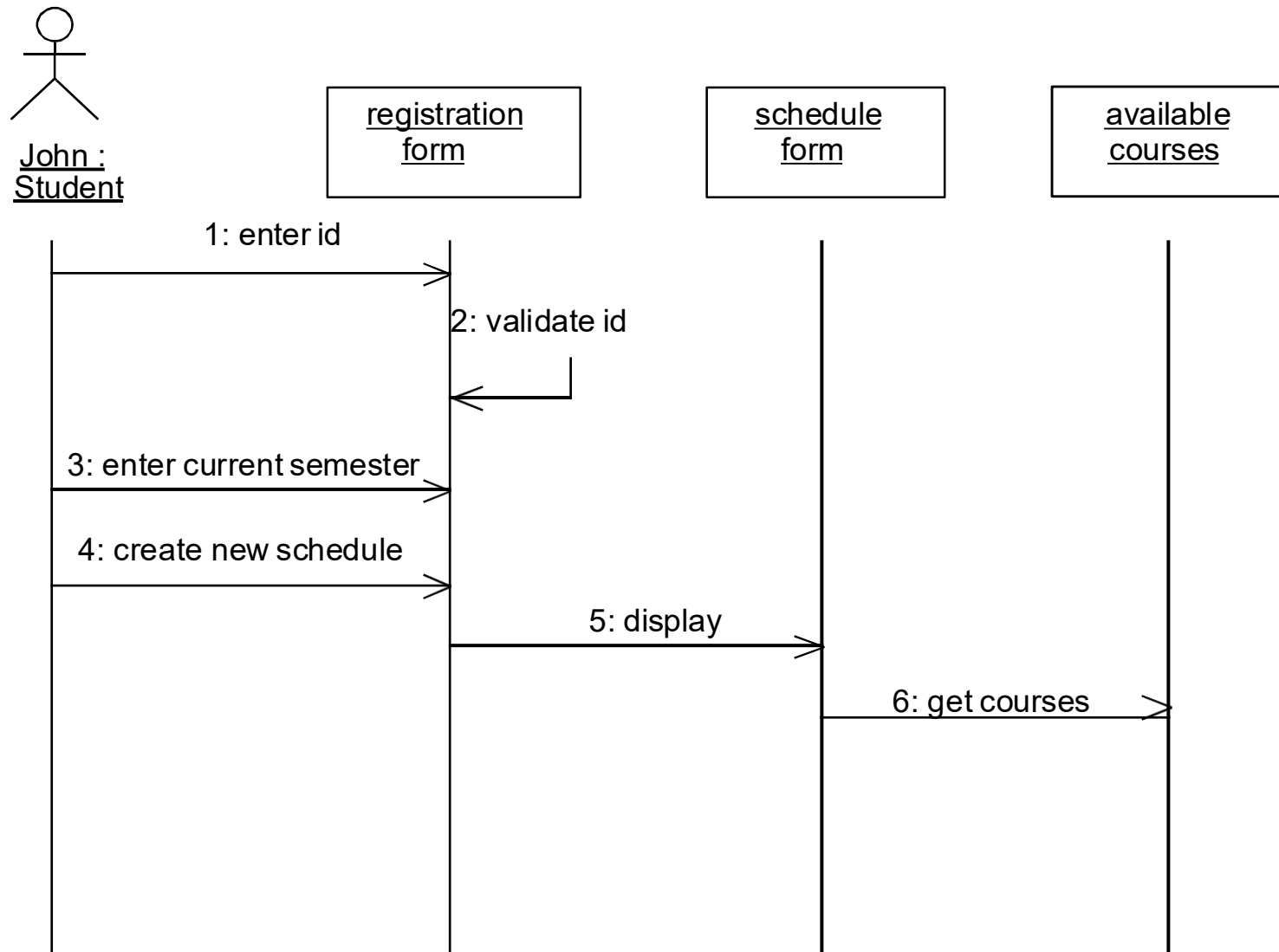
- Scenarios are typically captured in interaction diagrams as opposed to text
- There are two kinds of interaction diagrams
 - Sequence diagrams
 - Collaboration diagrams
- Each provides a different view of the same interaction
 - Sequence diagrams are time ordered
 - Collaboration diagrams may include data flow



Sequence Diagram

- **Sequence diagram** represents the messages exchanged by a set of objects during a scenario
- A sequence diagram contains:
 - Objects with their “lifelines”
 - Messages exchanged between objects **in ordered sequence**
 - Focus of control (optional)

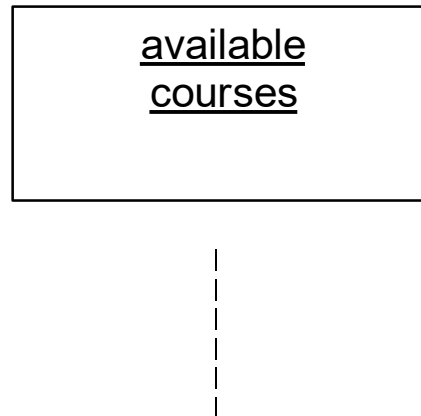
A Sequence Diagram





Naming Objects in Sequence Diagrams

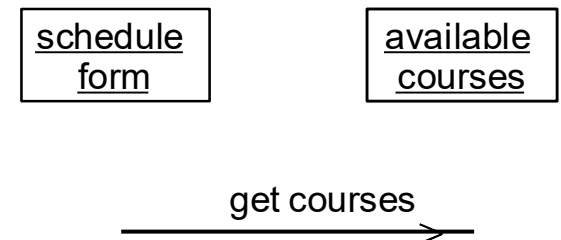
- Objects are drawn as rectangles with underlined names
- Object “lifelines” are represented by descending dashed lines





Showing Object Interaction

- Object interaction is indicated by horizontal arrows which are directed from the **client** object to the **supplier** object
- Horizontal arrows are labeled with message
- Time ordering of events is indicated by vertical position, with the earliest appearing at the top

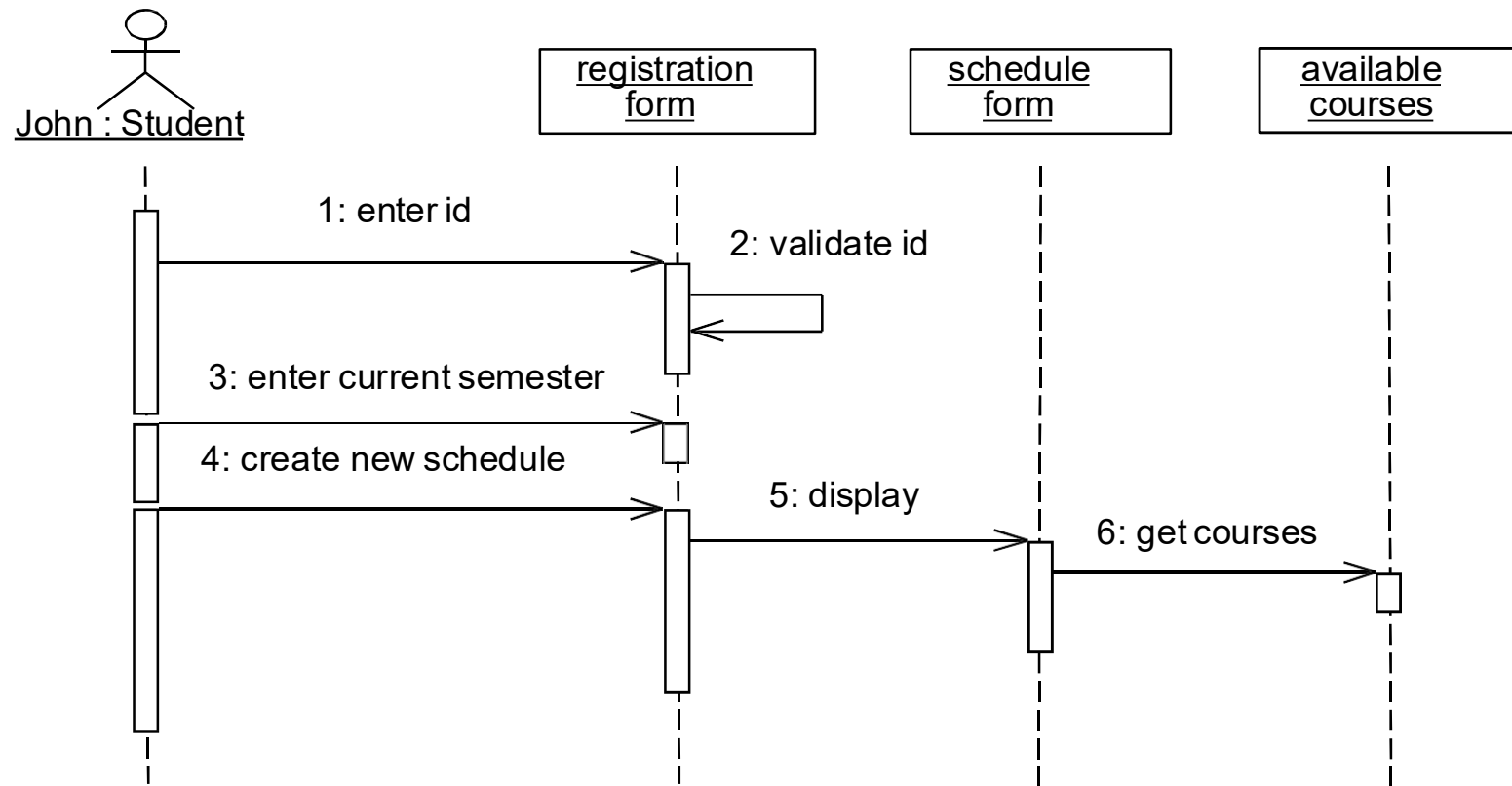




Focus of Control

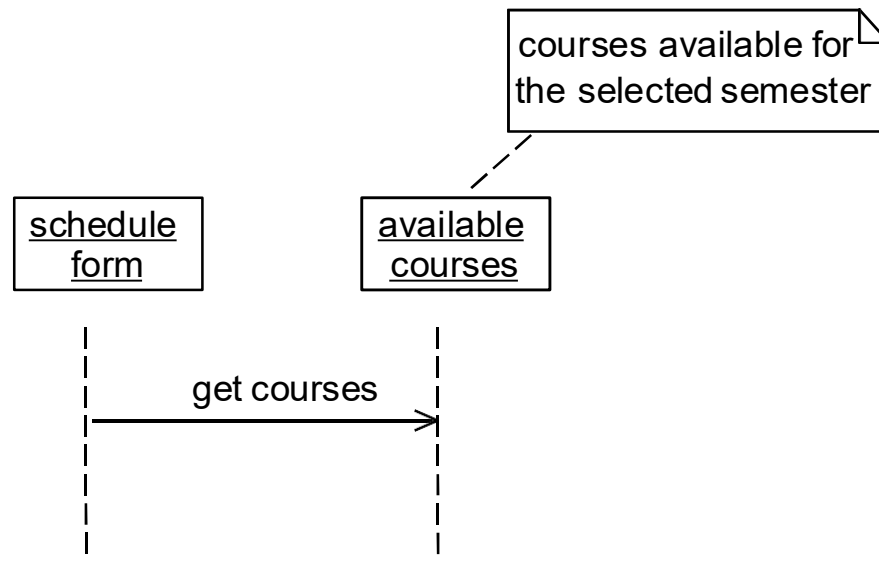
- Focus of Control represents the relative time that the flow of control is focused in an object
 - It represents the time an object is directing messages
- Focus of Control may be shown on a sequence diagram

Focus of Control



Notes

- Notes may be added to add more information to the diagram



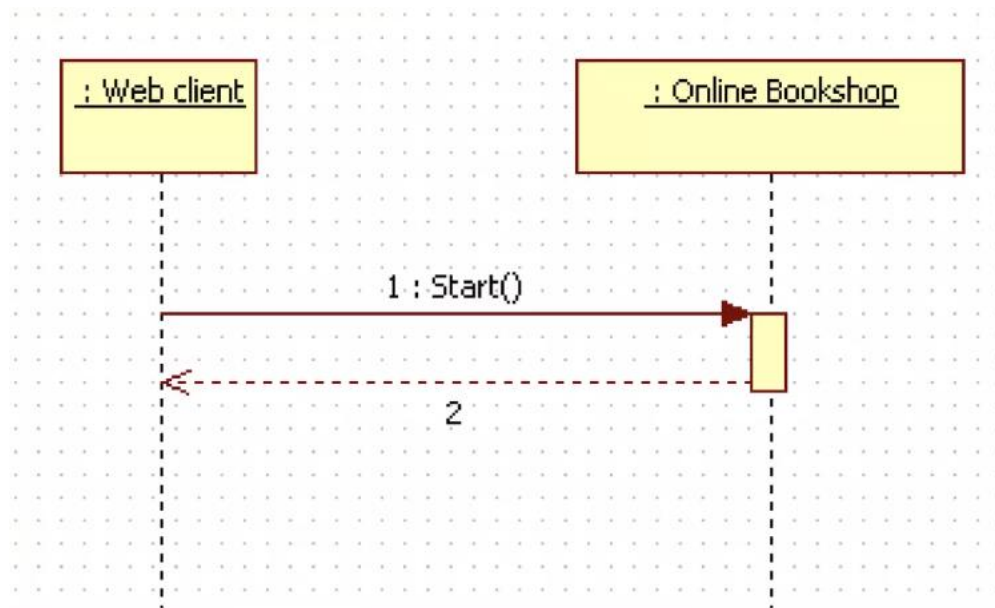


Scripts for Sequence Diagrams

- For complex scenarios, sequence diagrams can be enhanced by the use of scripts
- A script is written at the left of a sequence diagram with the steps of the script aligning with the object interactions
- Scripts can be written in natural language or pseudo code

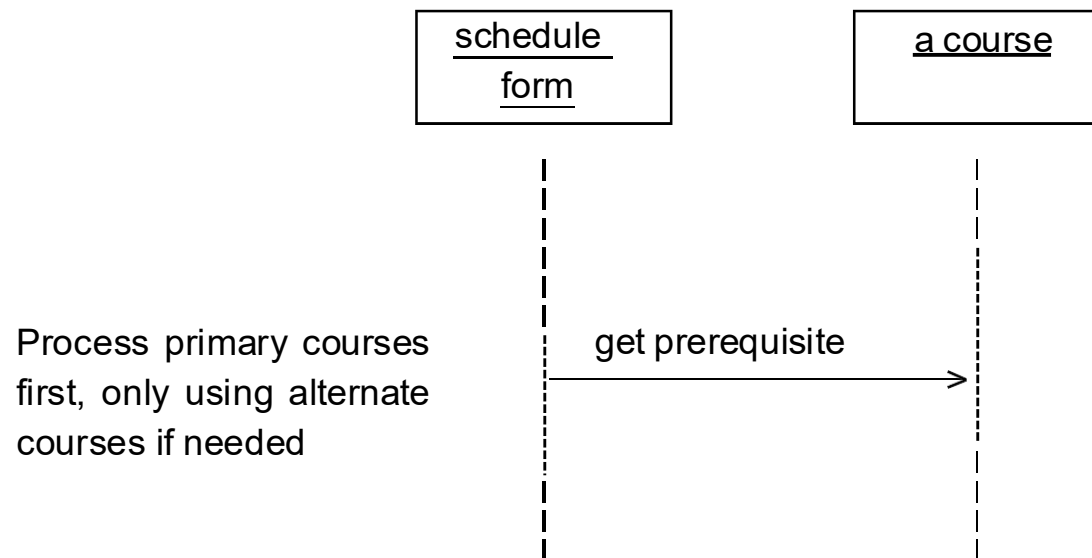


Send and Return Message





Sample Script





Example

alt- Alternative fragment models **if...then...else** constructs. Only one sequence occurs on any occasion.

opt- Optional. Encloses a sequence that might or might not happen. You can specify, in the guard, the condition under which it occurs.

break- If this fragment is executed, the rest of the sequence is abandoned.

Par- Parallel fragment denotes concurrent processing.



Example

Seq - There are two or more operand fragments. Messages involving the same lifeline must occur in the order of the fragments. Where they do not involve the same lifelines, messages from different fragments may be interleaved in parallel.

strict - Strict sequencing fragment encloses a series of messages which must occur in the given order.

neg - Negative fragment encloses an invalid series of messages.

loop - Loop fragment encloses a series of messages which are repeated.

Loop combined fragments have the properties Min and Max, which indicate the minimum and maximum number of times that the fragment can be repeated. The default is no restriction.



Example

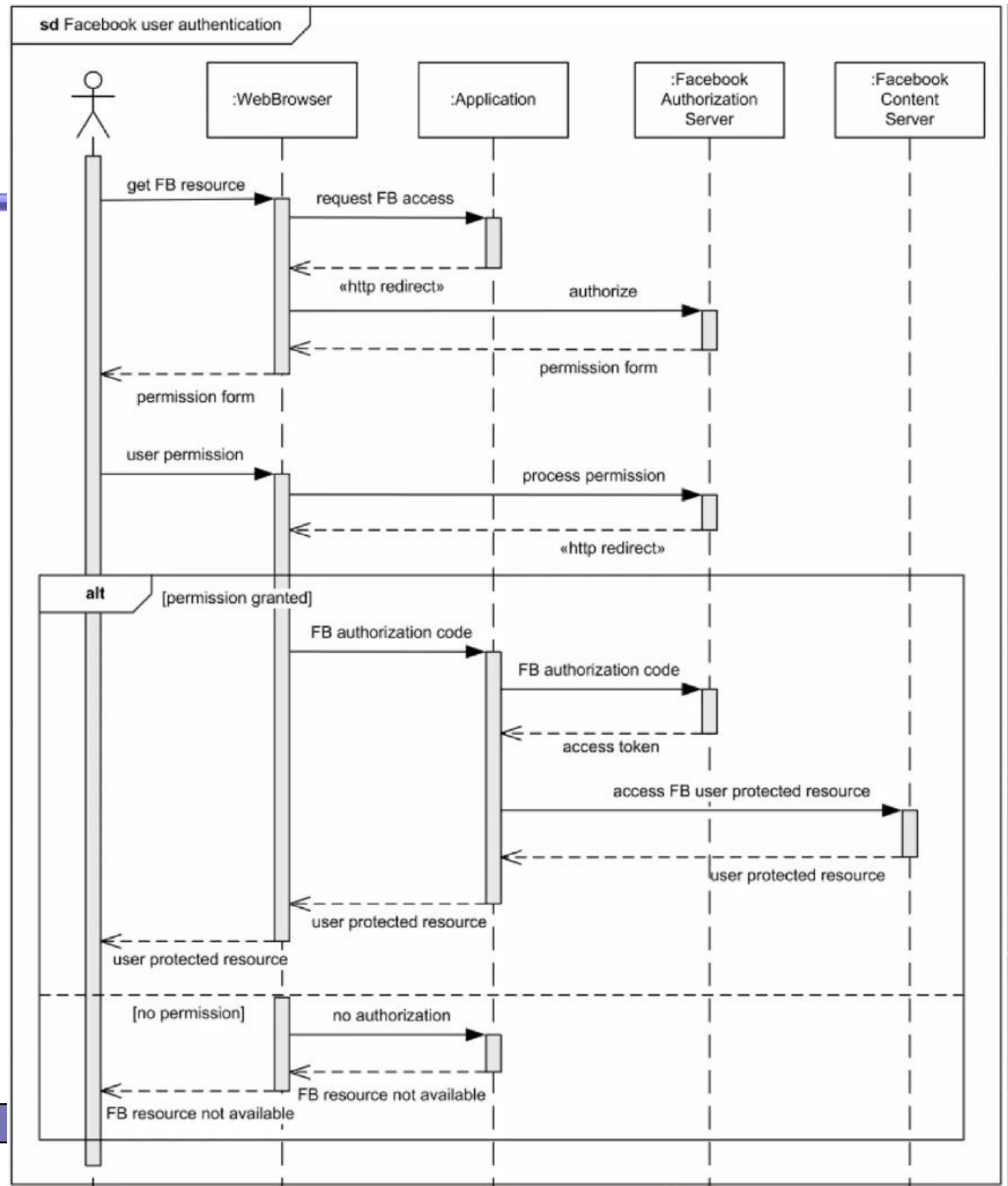
ignore - Ignore fragment declares a message or message to be of no interest if it appears in the current context.

consider - Consider fragment is in effect the opposite of the ignore fragment: any message not included in the consider fragment should be ignored.

assert - Assertion fragment designates that any sequence not shown as an operand of the assertion is invalid.



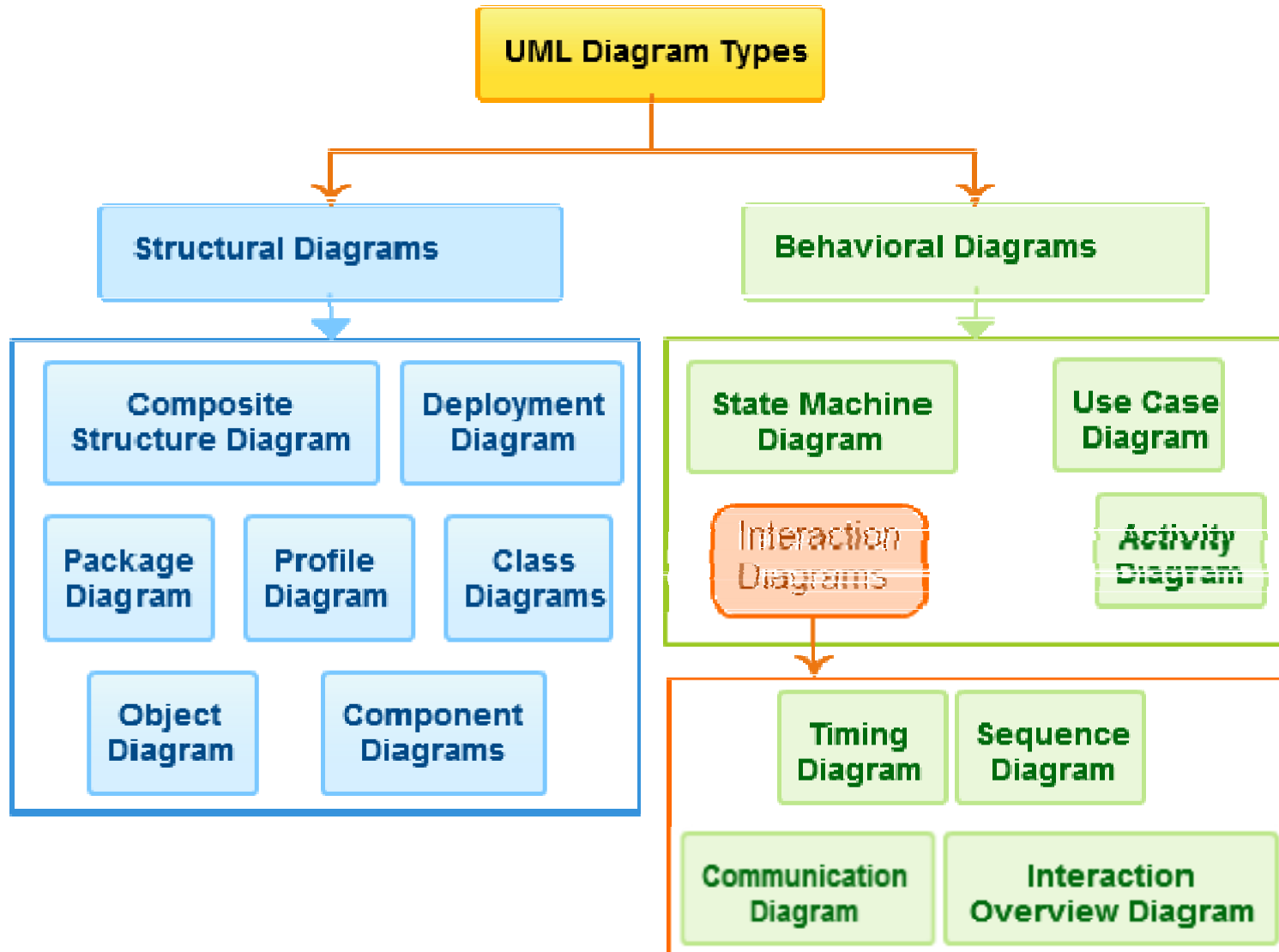
Example





sequence design exercise

UML Diagram types





Thank You

A thick blue horizontal bar at the bottom of the slide.